projet_climat

October 31, 2025

1 RAPPORT D'ANALYSE DE L'EVOLUTION CLIMATIQUE

1.1 Problématique

1.1.1 Comment les émissions de CO et le changement du climat affecte-t-ils la performance économique, la résilience agricole et la condition de vie au TOGO au fil du temps ?

Nous avons importer 5 datasets : température global par ville , temperature global par pays , TOGO DATA, temperature global , emissions du CO2 par pays .

SOURCES: https://donnees.banquemondiale.org/pays/togo https://www.kaggle.com/datasets/berkeleyearth/clin change-earth-surface-temperature-data https://www.kaggle.com/datasets/berkeleyearth/climate-change-earth-surface-temperature-data

- Utilisation des librairies python : pandas , matplotlib , seaborn ,numpy
- Lire les datasets
- Nettoyage des datasets (doublons , valeurs manquantes , formatage , statistiques descriptives)
- Analyse des datasets : Poser des hypothèses , vérifier ces hypothèses par des calculs de corrélations en déterminant les indicateurs appropriés , visualiser les résultats , interpréter les résultats et pour finir confirmer ou réfuter les hypothèses

Notre analyse s'articule autour de trois axes principaux : l'économie, l'agriculture et les conditions de vie. La problématique que nous explorons est la suivante : Comment les émissions de CO et les évolutions climatiques influencent-elles la performance économique, la résilience agricole et les conditions de vie au Togo au fil du temps ? Pour répondre à cette question, nous avons mobilisé trois sources de données principales :

Un jeu de données sur les émissions de CO , filtré spécifiquement sur le Togo ; Un jeu de données sur les températures moyennes par pays, également restreint au Togo ; Le jeu de données du Togo issu de la Banque mondiale, à partir duquel nous avons sélectionné des indicateurs pertinents en lien avec nos trois axes d'étude.

[231]:
```
# Importation des librairies
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

[232]:
```
# Chargement des données de température global par ville
df_temp = pd.read_csv("GlobalLandTemperaturesByCity.csv")
```

[233]:
```python
# Aperçu des données de température global par ville
df_temp.head()
```

[233]:
```
           dt  AverageTemperature  AverageTemperatureUncertainty   City  \
0  1743-11-01               6.068                          1.737  Århus
1  1743-12-01                 NaN                            NaN  Århus
2  1744-01-01                 NaN                            NaN  Århus
3  1744-02-01                 NaN                            NaN  Århus
4  1744-03-01                 NaN                            NaN  Århus

   Country Latitude Longitude
0  Denmark   57.05N    10.33E
1  Denmark   57.05N    10.33E
2  Denmark   57.05N    10.33E
3  Denmark   57.05N    10.33E
4  Denmark   57.05N    10.33E
```

[234]:
```python
# Chargement des données des émissions de CO2 par pays
df_c02 = pd.read_csv("API_EN.GHG.CO2.PC.CE.AR5_DS2_en_csv_v2_127841.csv")
```

[235]:
```python
# Aperçu des données des émissions de CO2 par pays
df_c02.head()
```

[235]:
```
                  Country Name Country Code  \
0                        Aruba          ABW
1  Africa Eastern and Southern          AFE
2                  Afghanistan          AFG
3   Africa Western and Central          AFW
4                       Angola          AGO

                                      Indicator Name        Indicator Code  \
0  Carbon dioxide (CO2) emissions excluding LULUC…  EN.GHG.CO2.PC.CE.AR5
1  Carbon dioxide (CO2) emissions excluding LULUC…  EN.GHG.CO2.PC.CE.AR5
2  Carbon dioxide (CO2) emissions excluding LULUC…  EN.GHG.CO2.PC.CE.AR5
3  Carbon dioxide (CO2) emissions excluding LULUC…  EN.GHG.CO2.PC.CE.AR5
4  Carbon dioxide (CO2) emissions excluding LULUC…  EN.GHG.CO2.PC.CE.AR5

   1960  1961  1962  1963  1964  1965  …      2016      2017      2018  \
0   NaN   NaN   NaN   NaN   NaN   NaN  …  4.459794  4.291167  4.277923
1   NaN   NaN   NaN   NaN   NaN   NaN  …  1.026116  1.012873  0.997157
2   NaN   NaN   NaN   NaN   NaN   NaN  …  0.216930  0.226011  0.215878
3   NaN   NaN   NaN   NaN   NaN   NaN  …  0.506323  0.495060  0.509713
4   NaN   NaN   NaN   NaN   NaN   NaN  …  1.072053  0.924169  0.839019

       2019      2020      2021      2022      2023  2024  Unnamed: 69
0  5.108834  4.168086  4.648097  4.684559  4.936708   NaN          NaN
1  0.983492  0.845417  0.863533  0.816361  0.784641   NaN          NaN
```

```
2  0.191491  0.180555  0.198268  0.203552  0.210043   NaN          NaN
3  0.518890  0.497159  0.512749  0.505462  0.482217   NaN          NaN
4  0.851665  0.619139  0.731567  0.767587  0.768163   NaN          NaN

[5 rows x 70 columns]
```

[236]:
```
# Chargement des données du togo extrait de la banque mondiale
df_togo = pd.read_csv("climate-change_tgo.csv")
```

[237]:
```
# Aperçu des données du togo extrait de la banque mondiale
df_togo.head()
```

[237]:
```
    Country Name   Country ISO3        Year              Indicator Name  \
0  #country+name  #country+code  #date+year            #indicator+name
1           Togo            TGO        2021  Agricultural land (sq. km)
2           Togo            TGO        2020  Agricultural land (sq. km)
3           Togo            TGO        2019  Agricultural land (sq. km)
4           Togo            TGO        2018  Agricultural land (sq. km)

    Indicator Code                 Value
0  #indicator+code  #indicator+value+num
1   AG.LND.AGRI.K2                 38200
2   AG.LND.AGRI.K2                 38200
3   AG.LND.AGRI.K2                 38200
4   AG.LND.AGRI.K2                 38200
```

[238]:
```
# Chargement des données de température global par pays
df_temp_count= pd.read_csv("GlobalLandTemperaturesByCountry.csv")
```

[239]:
```
# Chargement des données de température global
df_temp_glob = pd.read_csv("GlobalTemperatures (1).csv")
```

2 NETTOYAGE DU DATASET TEMPERATURE CITY

[240]:
```
#afficher les dimensions du dataset temperature par ville
print(f"Dimensions du dataset temperature: {df_temp.shape}")
```

```
Dimensions du dataset temperature: (8599212, 7)
```

[241]:
```
#afficher les doublons du dataset temperature par ville
duplicates_temp = df_temp[df_temp.duplicated()]
print(f"Nombre de doublons dans le dataset temperature: {len(duplicates_temp)}")
```

```
Nombre de doublons dans le dataset temperature: 0
```

[242]:
```
#afficher les valeurs manquantes dans le dataset temperature par ville
missing_temp = df_temp.isnull().sum()
print("Valeurs manquantes dans le dataset temperature:")
```

```
print(missing_temp)
```

```
Valeurs manquantes dans le dataset temperature:
dt                                     0
AverageTemperature                364130
AverageTemperatureUncertainty     364130
City                                   0
Country                                0
Latitude                               0
Longitude                              0
dtype: int64
```

[243]:
```
#supprimer les valeurs manquantes AverageTemperature et␣
↪AverageTemperatureUncertainty
df_temp_cleaned = df_temp.dropna(subset=['AverageTemperature',␣
↪'AverageTemperatureUncertainty'])
print(f"Dimensions du dataset temperature après nettoyage: {df_temp_cleaned.
↪shape}")
```

```
Dimensions du dataset temperature après nettoyage: (8235082, 7)
```

[244]:
```
#afficher les valeurs manquantes dans le dataset temperature après nettoyage
print(df_temp_cleaned.isnull().sum())
```

```
dt                                 0
AverageTemperature                 0
AverageTemperatureUncertainty      0
City                               0
Country                            0
Latitude                           0
Longitude                          0
dtype: int64
```

[245]:
```
#verifier le type des colonnes
print(df_temp_cleaned.dtypes)
```

```
dt                                  object
AverageTemperature                 float64
AverageTemperatureUncertainty      float64
City                                object
Country                             object
Latitude                            object
Longitude                           object
dtype: object
```

[246]:
```
#convertir la colonne dt en type datetime
df_temp_cleaned["dt"] = pd.to_datetime(df_temp_cleaned["dt"])
```

```
C:\Users\fangn\AppData\Local\Temp\ipykernel_15492\1921518905.py:2:
SettingWithCopyWarning:
```

```
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df_temp_cleaned["dt"] = pd.to_datetime(df_temp_cleaned["dt"])
```

[247]:
```python
#convertir les colonnes Latitude et Longitude en format numérique
def convert_coord(coord):
    try:
        coord = str(coord)
        if coord[-1] in ["N", "E"]:
            return float(coord[:-1])
        elif coord[-1] in ["S", "W"]:
            return -float(coord[:-1])
        else:
            return float(coord)  # au cas où il n'y a pas de lettre
    except:
        return None  # pour les NaN ou formats inattendus

df_temp_cleaned["Latitude"] = df_temp_cleaned["Latitude"].apply(convert_coord)
df_temp_cleaned["Longitude"] = df_temp_cleaned["Longitude"].apply(convert_coord)
```

```
C:\Users\fangn\AppData\Local\Temp\ipykernel_15492\1794684259.py:14:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df_temp_cleaned["Latitude"] = df_temp_cleaned["Latitude"].apply(convert_coord)
C:\Users\fangn\AppData\Local\Temp\ipykernel_15492\1794684259.py:15:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df_temp_cleaned["Longitude"] =
df_temp_cleaned["Longitude"].apply(convert_coord)
```

[248]:
```python
#afficher les statistiques descriptives du dataset temperature
print("Statistiques descriptives du dataset temperature:")
print(df_temp_cleaned.describe())
```

```
Statistiques descriptives du dataset temperature:
                                 dt  AverageTemperature  \
count                        8235082        8.235082e+06
mean   1911-04-17 10:54:53.185130496        1.672743e+01
```

```
min              1743-11-01 00:00:00       -4.270400e+01
25%              1865-09-01 00:00:00        1.029900e+01
50%              1915-12-01 00:00:00        1.883100e+01
75%              1964-10-01 00:00:00        2.521000e+01
max              2013-09-01 00:00:00        3.965100e+01
std                              NaN        1.035344e+01

       AverageTemperatureUncertainty      Latitude     Longitude
count                   8.235082e+06  8.235082e+06  8.235082e+06
mean                    1.028575e+00  2.609259e+01  3.013611e+01
min                     3.400000e-02 -5.224000e+01 -1.511300e+02
25%                     3.370000e-01  1.366000e+01 -4.470000e+00
50%                     5.910000e-01  3.135000e+01  3.169000e+01
75%                     1.349000e+00  4.259000e+01  8.825000e+01
max                     1.539600e+01  6.992000e+01  1.769500e+02
std                     1.129733e+00  2.280415e+01  7.236688e+01
```

[249]:
```python
#extraire le dataset  city nettoyé en csv
df_temp_cleaned.to_csv("GlobalLandTemperaturesByCity_cleaned.csv", index=False)
```

3 NETTOYAGE DU DATASET CO2

[250]:
```python
df_c02.head()
```

[250]:
```
                  Country Name Country Code  \
0                        Aruba          ABW
1  Africa Eastern and Southern          AFE
2                  Afghanistan          AFG
3   Africa Western and Central          AFW
4                       Angola          AGO

                                     Indicator Name        Indicator Code  \
0  Carbon dioxide (CO2) emissions excluding LULUC…  EN.GHG.CO2.PC.CE.AR5
1  Carbon dioxide (CO2) emissions excluding LULUC…  EN.GHG.CO2.PC.CE.AR5
2  Carbon dioxide (CO2) emissions excluding LULUC…  EN.GHG.CO2.PC.CE.AR5
3  Carbon dioxide (CO2) emissions excluding LULUC…  EN.GHG.CO2.PC.CE.AR5
4  Carbon dioxide (CO2) emissions excluding LULUC…  EN.GHG.CO2.PC.CE.AR5

   1960  1961  1962  1963  1964  1965  …      2016      2017      2018  \
0   NaN   NaN   NaN   NaN   NaN   NaN  …  4.459794  4.291167  4.277923
1   NaN   NaN   NaN   NaN   NaN   NaN  …  1.026116  1.012873  0.997157
2   NaN   NaN   NaN   NaN   NaN   NaN  …  0.216930  0.226011  0.215878
3   NaN   NaN   NaN   NaN   NaN   NaN  …  0.506323  0.495060  0.509713
4   NaN   NaN   NaN   NaN   NaN   NaN  …  1.072053  0.924169  0.839019

       2019      2020      2021      2022      2023  2024  Unnamed: 69
0  5.108834  4.168086  4.648097  4.684559  4.936708   NaN          NaN
```

```
1  0.983492  0.845417  0.863533  0.816361  0.784641   NaN          NaN
2  0.191491  0.180555  0.198268  0.203552  0.210043   NaN          NaN
3  0.518890  0.497159  0.512749  0.505462  0.482217   NaN          NaN
4  0.851665  0.619139  0.731567  0.767587  0.768163   NaN          NaN

[5 rows x 70 columns]
```

[251]:
```
df_c02.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 266 entries, 0 to 265
Data columns (total 70 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Country Name    266 non-null    object
 1   Country Code    266 non-null    object
 2   Indicator Name  266 non-null    object
 3   Indicator Code  266 non-null    object
 4   1960            0 non-null      float64
 5   1961            0 non-null      float64
 6   1962            0 non-null      float64
 7   1963            0 non-null      float64
 8   1964            0 non-null      float64
 9   1965            0 non-null      float64
 10  1966            0 non-null      float64
 11  1967            0 non-null      float64
 12  1968            0 non-null      float64
 13  1969            0 non-null      float64
 14  1970            251 non-null    float64
 15  1971            251 non-null    float64
 16  1972            251 non-null    float64
 17  1973            251 non-null    float64
 18  1974            251 non-null    float64
 19  1975            251 non-null    float64
 20  1976            251 non-null    float64
 21  1977            251 non-null    float64
 22  1978            251 non-null    float64
 23  1979            251 non-null    float64
 24  1980            251 non-null    float64
 25  1981            251 non-null    float64
 26  1982            251 non-null    float64
 27  1983            251 non-null    float64
 28  1984            251 non-null    float64
 29  1985            251 non-null    float64
 30  1986            251 non-null    float64
 31  1987            251 non-null    float64
 32  1988            251 non-null    float64
 33  1989            251 non-null    float64
```

```
 34  1990            251 non-null    float64
 35  1991            251 non-null    float64
 36  1992            251 non-null    float64
 37  1993            251 non-null    float64
 38  1994            251 non-null    float64
 39  1995            251 non-null    float64
 40  1996            251 non-null    float64
 41  1997            251 non-null    float64
 42  1998            251 non-null    float64
 43  1999            251 non-null    float64
 44  2000            251 non-null    float64
 45  2001            251 non-null    float64
 46  2002            251 non-null    float64
 47  2003            251 non-null    float64
 48  2004            251 non-null    float64
 49  2005            251 non-null    float64
 50  2006            251 non-null    float64
 51  2007            251 non-null    float64
 52  2008            251 non-null    float64
 53  2009            251 non-null    float64
 54  2010            251 non-null    float64
 55  2011            251 non-null    float64
 56  2012            251 non-null    float64
 57  2013            251 non-null    float64
 58  2014            251 non-null    float64
 59  2015            251 non-null    float64
 60  2016            251 non-null    float64
 61  2017            251 non-null    float64
 62  2018            251 non-null    float64
 63  2019            251 non-null    float64
 64  2020            251 non-null    float64
 65  2021            251 non-null    float64
 66  2022            251 non-null    float64
 67  2023            251 non-null    float64
 68  2024            0 non-null      float64
 69  Unnamed: 69     0 non-null      float64
dtypes: float64(66), object(4)
memory usage: 145.6+ KB
```

[252]:
```
#afficher les doublons du datset co2
duplicates_co2 = df_c02[df_c02.duplicated()]
print(f"Nombre de doublons dans le dataset co2: {len(duplicates_co2)}")
```

```
Nombre de doublons dans le dataset co2: 0
```

[253]:
```
#afficher les valeurs manquantes dans le dataset temperature CO2
print(f"Dimensions du dataset CO2 après nettoyage : {df_c02.shape}")
print(df_c02.isna().sum())
```

```
Dimensions du dataset CO2 après nettoyage : (266, 70)
Country Name        0
Country Code        0
Indicator Name      0
Indicator Code      0
1960              266
                 …
2021               15
2022               15
2023               15
2024              266
Unnamed: 69       266
Length: 70, dtype: int64
```

[254]:
```
#extraire le datset co2 nettoyé
df_c02.to_csv("co2_cleaned.csv", index=False)
```

4 NETTOYAGE DU DATASET TOGO

[255]:
```
df_togo.head()
```

[255]:
```
     Country Name   Country ISO3        Year              Indicator Name  \
0  #country+name  #country+code  #date+year            #indicator+name
1           Togo            TGO        2021  Agricultural land (sq. km)
2           Togo            TGO        2020  Agricultural land (sq. km)
3           Togo            TGO        2019  Agricultural land (sq. km)
4           Togo            TGO        2018  Agricultural land (sq. km)

    Indicator Code                 Value
0  #indicator+code  #indicator+value+num
1   AG.LND.AGRI.K2                 38200
2   AG.LND.AGRI.K2                 38200
3   AG.LND.AGRI.K2                 38200
4   AG.LND.AGRI.K2                 38200
```

[256]:
```
#afficher les doublons du datset togo
duplicates_togo = df_togo[df_togo.duplicated()]
print(f"Nombre de doublons dans le dataset togo: {len(duplicates_togo)}")
```

```
Nombre de doublons dans le dataset togo: 0
```

[257]:
```
#afficher les valeurs manquantes dans le dataset temperature
missing_togo = df_togo.isnull().sum()
print("Valeurs manquantes dans le dataset temperature:")
print(missing_togo)
```

```
Valeurs manquantes dans le dataset temperature:
Country Name      0
```

```
Country ISO3      0
Year              0
Indicator Name    0
Indicator Code    0
Value             0
dtype: int64
```

5 NETTOYAGE DU DATASET TEMPERATURE COUNTRY

[258]:
```python
#afficher l'entête du dataset temperature country
df_temp_count.head()
```

[258]:

	dt	AverageTemperature	AverageTemperatureUncertainty	Country
0	1743-11-01	4.384	2.294	Åland
1	1743-12-01	NaN	NaN	Åland
2	1744-01-01	NaN	NaN	Åland
3	1744-02-01	NaN	NaN	Åland
4	1744-03-01	NaN	NaN	Åland

[259]:
```python
#afficher les dimensions du datset temperature country
print(f"Dimensions du dataset temperature country: {df_temp_count.shape}")
```

```
Dimensions du dataset temperature country: (577462, 4)
```

[260]:
```python
#afficher les doublons du dataset temperature country
duplicates_temp_count = df_temp_count[df_temp_count.duplicated()]
print(f"Nombre de doublons dans le dataset temperature country:␣
↪{len(duplicates_temp_count)}")
```

```
Nombre de doublons dans le dataset temperature country: 0
```

[261]:
```python
#afficher les valeurs manquantes dans le dataset temperature country
missing_temp_count = df_temp_count.isnull().sum()
print("Valeurs manquantes dans le dataset temperature country:")
print(missing_temp_count)
```

```
Valeurs manquantes dans le dataset temperature country:
dt                                  0
AverageTemperature              32651
AverageTemperatureUncertainty   31912
Country                             0
dtype: int64
```

[262]:
```python
#supprimer les valeurs manquantes AverageTemperature et␣
↪AverageTemperatureUncertainty
df_temp_count_cleaned = df_temp_count.dropna(subset=['AverageTemperature',␣
↪'AverageTemperatureUncertainty'])
```

```
print(f"Dimensions du dataset temperature country après nettoyage:␣
↪{df_temp_count_cleaned.shape}")
```

```
Dimensions du dataset temperature country après nettoyage: (544811, 4)
```

[263]:
```
#afficher les valeurs manquantes après nettoyage dans le dataset temperature␣
↪country
print("Valeurs manquantes dans le dataset temperature country après nettoyage:")
print(df_temp_count_cleaned.isnull().sum())
```

```
Valeurs manquantes dans le dataset temperature country après nettoyage:
dt                              0
AverageTemperature              0
AverageTemperatureUncertainty   0
Country                         0
dtype: int64
```

[264]:
```
#afficher les statistiques descriptives du dataset temperature country
print("Statistiques descriptives du dataset temperature country:")
print(df_temp_count_cleaned.describe())
```

```
Statistiques descriptives du dataset temperature country:
       AverageTemperature  AverageTemperatureUncertainty
count       544811.000000                  544811.000000
mean            17.193354                       1.019190
std             10.953966                       1.202634
min            -37.658000                       0.052000
25%             10.025000                       0.323000
50%             20.901000                       0.571000
75%             25.814000                       1.207000
max             38.842000                      15.003000
```

[265]:
```
#afficher le type des colonnes du dataset temperature country
print(df_temp_count_cleaned.dtypes)
```

```
dt                              object
AverageTemperature             float64
AverageTemperatureUncertainty  float64
Country                         object
dtype: object
```

[266]:
```
#convertir la colonne dt en type datetime
df_temp_count_cleaned["dt"] = pd.to_datetime(df_temp_count_cleaned["dt"])
```

```
C:\Users\fangn\AppData\Local\Temp\ipykernel_15492\3951450442.py:2:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
```

```
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df_temp_count_cleaned["dt"] = pd.to_datetime(df_temp_count_cleaned["dt"])
```

[267]:
```
#extraire le dataset  country nettoyé en csv
df_temp_count_cleaned.to_csv("GlobalLandTemperaturesByCountry_cleaned.csv",␣
 ↪index=False)
```

6 NETTOYAGE DU DATASET TEMPERATURE GLOBAL

[268]:
```
#afficher l'entête du dataset global temperatures
df_temp_glob.head()
```

[268]:
```
           dt  LandAverageTemperature  LandAverageTemperatureUncertainty  \
0  1750-01-01                   3.034                              3.574
1  1750-02-01                   3.083                              3.702
2  1750-03-01                   5.626                              3.076
3  1750-04-01                   8.490                              2.451
4  1750-05-01                  11.573                              2.072

   LandMaxTemperature  LandMaxTemperatureUncertainty  LandMinTemperature  \
0                 NaN                            NaN                 NaN
1                 NaN                            NaN                 NaN
2                 NaN                            NaN                 NaN
3                 NaN                            NaN                 NaN
4                 NaN                            NaN                 NaN

   LandMinTemperatureUncertainty  LandAndOceanAverageTemperature  \
0                            NaN                             NaN
1                            NaN                             NaN
2                            NaN                             NaN
3                            NaN                             NaN
4                            NaN                             NaN

   LandAndOceanAverageTemperatureUncertainty
0                                        NaN
1                                        NaN
2                                        NaN
3                                        NaN
4                                        NaN
```

[269]:
```
#affciher les doublons du dataset global temperatures
duplicates_temp_glob = df_temp_glob[df_temp_glob.duplicated()]
print(f"Nombre de doublons dans le dataset global temperatures:␣
 ↪{len(duplicates_temp_glob)}")
```

```
Nombre de doublons dans le dataset global temperatures: 0
```

[270]:
```
#afficher le dimensions du datset global temperatures
print(f"Dimensions du dataset global temperatures: {df_temp_glob.shape}")
```

```
Dimensions du dataset global temperatures: (3192, 9)
```

[271]:
```
#afficher les valeurs manquantes dans le dataset temperature
missing_temp = df_temp_glob.isnull().sum()
print("Valeurs manquantes dans le dataset temperature global:")
print(missing_temp)
```

```
Valeurs manquantes dans le dataset temperature global:
dt                                            0
LandAverageTemperature                       12
LandAverageTemperatureUncertainty            12
LandMaxTemperature                         1200
LandMaxTemperatureUncertainty              1200
LandMinTemperature                         1200
LandMinTemperatureUncertainty              1200
LandAndOceanAverageTemperature             1200
LandAndOceanAverageTemperatureUncertainty  1200
dtype: int64
```

[272]:
```
cols_to_clean = [
    "LandAverageTemperature",
    "LandAverageTemperatureUncertainty",
    "LandMaxTemperature",
    "LandMaxTemperatureUncertainty",
    "LandMinTemperature",
    "LandMinTemperatureUncertainty",
    "LandAndOceanAverageTemperature",
    "LandAndOceanAverageTemperatureUncertainty"
]


df_temp_glob_cleaned = df_temp_glob.copy()

for col in cols_to_clean:
    df_temp_glob_cleaned = df_temp_glob_cleaned[df_temp_glob_cleaned[col].
 ↪notna()]
```

[273]:
```
#afficher les valeurs manquantes après nettoyage dans le dataset temperature␣
 ↪global
print("Valeurs manquantes après nettoyage dans le dataset temperature global:")
print(df_temp_glob_cleaned.isnull().sum())
```

```
Valeurs manquantes après nettoyage dans le dataset temperature global:
dt                                          0
LandAverageTemperature                      0
LandAverageTemperatureUncertainty           0
```

```
LandMaxTemperature                            0
LandMaxTemperatureUncertainty                 0
LandMinTemperature                            0
LandMinTemperatureUncertainty                 0
LandAndOceanAverageTemperature                0
LandAndOceanAverageTemperatureUncertainty     0
dtype: int64
```

[274]:
```python
#afficher les statistiques descriptives du dataset temperature country
print("Statistiques descriptives du dataset temperature global:")
print(df_temp_glob_cleaned.describe())
```

```
Statistiques descriptives du dataset temperature global:
       LandAverageTemperature  LandAverageTemperatureUncertainty  \
count             1992.000000                        1992.000000
mean                 8.571583                           0.276663
std                  4.263193                           0.224030
min                  0.404000                           0.034000
25%                  4.430000                           0.099750
50%                  8.850500                           0.230000
75%                 12.858500                           0.347250
max                 15.482000                           1.492000

       LandMaxTemperature  LandMaxTemperatureUncertainty  LandMinTemperature  \
count         1992.000000                     1992.000000         1992.000000
mean            14.350601                        0.479782            2.743595
std              4.309579                        0.583203            4.155835
min              5.900000                        0.044000           -5.407000
25%             10.212000                        0.142000           -1.334500
50%             14.760000                        0.252000            2.949500
75%             18.451500                        0.539000            6.778750
max             21.320000                        4.373000            9.715000

       LandMinTemperatureUncertainty  LandAndOceanAverageTemperature  \
count                    1992.000000                     1992.000000
mean                        0.431849                       15.212566
std                         0.445838                        1.274093
min                         0.045000                       12.475000
25%                         0.155000                       14.047000
50%                         0.279000                       15.251000
75%                         0.458250                       16.396250
max                         3.498000                       17.611000

       LandAndOceanAverageTemperatureUncertainty
count                                1992.000000
mean                                    0.128532
std                                     0.073587
min                                     0.042000
```

```
25%                                     0.063000
50%                                     0.122000
75%                                     0.151000
max                                     0.457000
```

[275]:
```python
#extraire le dataset country nettoyé en csv
df_temp_glob_cleaned.to_csv("GlobalTemperatures_cleaned.csv", index=False)
```

LES FILTRES DES DONNEES PAR DATASET POUR LE TOGO

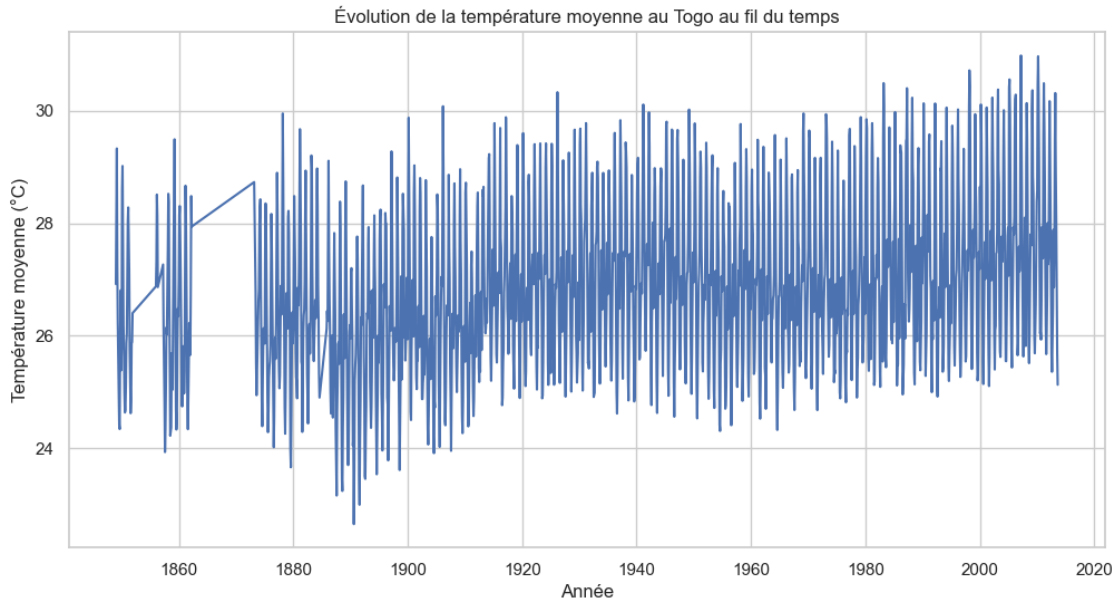
[276]:
```python
#afficher les informations pour le togo dans le dataset temperature country
df_togo_info = df_temp_count_cleaned[df_temp_count_cleaned["Country"] == "Togo"]
print("Informations pour le Togo dans le dataset temperature country:")
print(df_togo_info.shape)
df_togo_info.head()
```

```
Informations pour le Togo dans le dataset temperature country:
(1763, 4)
```

[276]:

	dt	AverageTemperature	AverageTemperatureUncertainty	Country
524539	1849-01-01	26.919	1.113	Togo
524540	1849-02-01	28.112	1.086	Togo
524541	1849-03-01	29.327	1.389	Togo
524542	1849-04-01	27.284	1.170	Togo
524543	1849-05-01	26.253	0.988	Togo

[277]:
```python
#visualiser l'évolution de la température moyenne au Togo au fil du temps avec␣
↪lineplot
togo_temp_over_time = df_togo_info.groupby("dt")["AverageTemperature"].mean().
↪reset_index()
plt.figure(figsize=(12, 6))
sns.lineplot(data=togo_temp_over_time, x="dt", y="AverageTemperature")
plt.title("Évolution de la température moyenne au Togo au fil du temps")
plt.xlabel("Année")
plt.ylabel("Température moyenne (°C)")
plt.show()
```



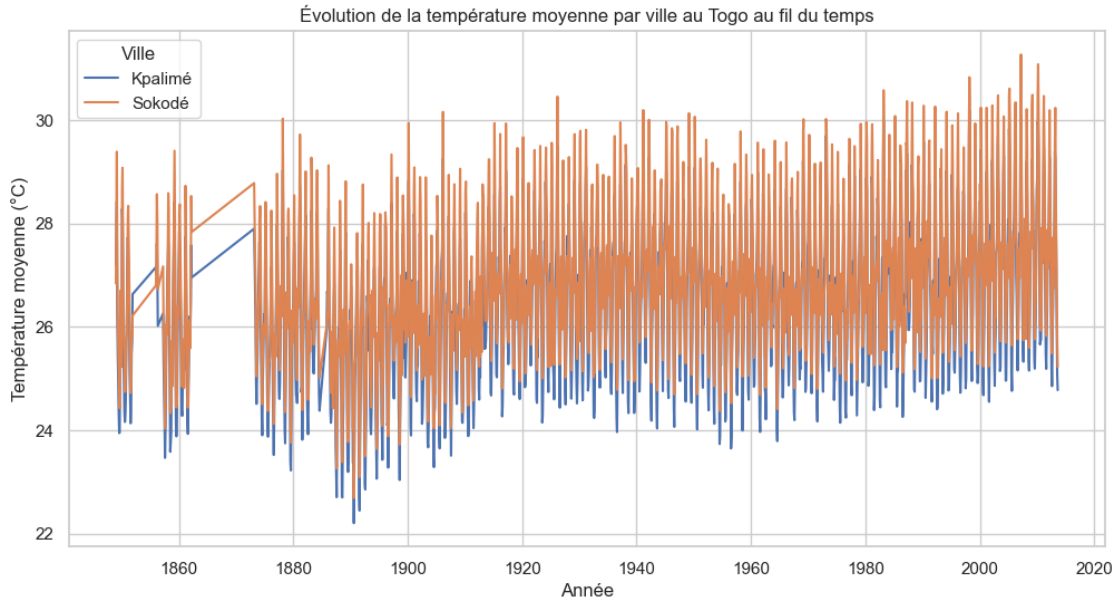
[278]:
```
#afficher les informations pour le togo dans le dataset temperature city
df_togo_info_city = df_temp_cleaned[df_temp_cleaned["Country"] == "Togo"]
print("Informations pour le Togo dans le dataset temperature city:")
print(df_togo_info_city.shape)
df_togo_info_city.head()
```

```
Informations pour le Togo dans le dataset temperature city:
(3526, 7)
```

[278]:
```
                dt  AverageTemperature  AverageTemperatureUncertainty  \
3981243 1849-01-01              27.194                          1.531
3981244 1849-02-01              27.792                          1.470
3981245 1849-03-01              28.421                          1.633
3981246 1849-04-01              26.280                          1.451
3981247 1849-05-01              25.453                          1.247

            City Country  Latitude  Longitude
3981243  Kpalimé    Togo      7.23       0.81
3981244  Kpalimé    Togo      7.23       0.81
3981245  Kpalimé    Togo      7.23       0.81
3981246  Kpalimé    Togo      7.23       0.81
3981247  Kpalimé    Togo      7.23       0.81
```

[279]:
```
#visualiser l'évolution de la température moyenne par ville  au fil du temps␣
↪avec lineplot
togo_temp_by_city = df_togo_info_city.groupby(["dt",␣
↪"City"])["AverageTemperature"].mean().reset_index()
```

```
plt.figure(figsize=(12, 6))
sns.lineplot(data=togo_temp_by_city, x="dt", y="AverageTemperature", hue="City")
plt.title("Évolution de la température moyenne par ville au Togo au fil du␣
 ↪temps")
plt.xlabel("Année")
plt.ylabel("Température moyenne (°C)")
plt.legend(title="Ville")
plt.show()
```



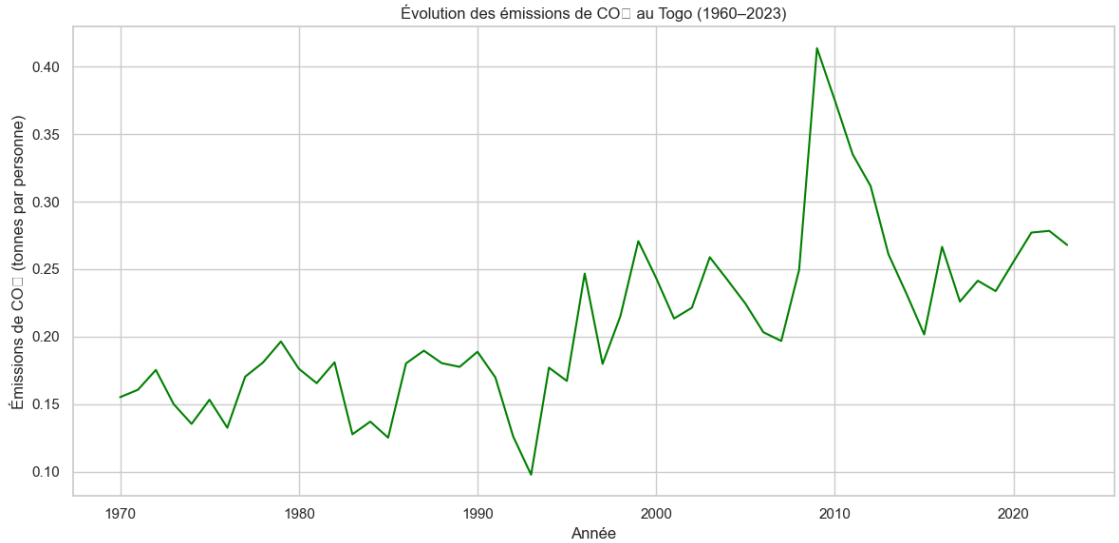
[280]:
```
#afficher les informations pour le togo dans le dataset co2
df_togo_info_co2 = df_c02[df_c02["Country Name"] == "Togo"]
print("Informations pour le Togo dans le dataset CO2:")
print(df_togo_info_co2.shape)
print(df_togo_info_co2)
```

```
Informations pour le Togo dans le dataset CO2:
(1, 70)
    Country Name Country Code  \
232         Togo          TGO

                                        Indicator Name        Indicator Code  \
232  Carbon dioxide (CO2) emissions excluding LULUC…  EN.GHG.CO2.PC.CE.AR5

     1960  1961  1962  1963  1964  1965  …      2016      2017      2018  \
232   NaN   NaN   NaN   NaN   NaN   NaN  …  0.266521  0.225936  0.241392

         2019      2020      2021      2022      2023  2024  Unnamed: 69
```

```
232  0.233698  0.255533  0.277078  0.278347  0.267897   NaN          NaN

[1 rows x 70 columns]
```

[281]:
```python
#visualisation de l'évolution de l'emission du CO2 au Togo au fil du temps de␣
↪1960 à 2023


# 1. Transformation du format large en format long
togo_co2_over_time = df_togo_info_co2.melt(
    id_vars=["Country Name", "Country Code", "Indicator Name", "Indicator␣
↪Code"],
    var_name="Year",
    value_name="CO2 Emissions"
)

# 2. Conversion de l'année en numérique
togo_co2_over_time["Year"] = pd.to_numeric(togo_co2_over_time["Year"],␣
↪errors='coerce')

# 3. Filtrer sur la période 1960-2023
togo_co2_over_time = togo_co2_over_time[
    (togo_co2_over_time["Year"] >= 1960) & (togo_co2_over_time["Year"] <= 2023)
]

# 4. Supprimer les lignes sans données d'émission
togo_co2_over_time = togo_co2_over_time.dropna(subset=["CO2 Emissions"])

# 5. Tracer la courbe
plt.figure(figsize=(12, 6))
sns.lineplot(data=togo_co2_over_time, x="Year", y="CO2 Emissions",␣
↪color="green")
plt.title("Évolution des émissions de CO  au Togo (1960-2023)")
plt.xlabel("Année")
plt.ylabel("Émissions de CO  (tonnes par personne)")
plt.grid(True)
plt.tight_layout()
plt.show()
```

```
C:\Users\fangn\AppData\Local\Temp\ipykernel_15492\353402376.py:29: UserWarning:
Glyph 8322 (\N{SUBSCRIPT TWO}) missing from font(s) Arial.
  plt.tight_layout()
c:\Users\fangn\OneDrive\Bureau\BACHELOR 2 NEXA DIGITAL SCHOOL\SEMESTRE
1\EVOLUTION DU CLIMAT  CERVELLO\env\Lib\site-
packages\IPython\core\pylabtools.py:170: UserWarning: Glyph 8322 (\N{SUBSCRIPT
TWO}) missing from font(s) Arial.
  fig.canvas.print_figure(bytes_io, **kw)
```



[282]:
```
#afficher les informations de l'agriculture pour le togo
df_togo_agri = df_togo[df_togo["Indicator Name"].str.contains("Agriculture",␣
↪na=False)]
print("Informations de l'agriculture pour le Togo:")
print(df_togo_agri.shape)
df_togo_agri
```

```
Informations de l'agriculture pour le Togo:
(62, 6)
```

[282]:
```
     Country Name Country ISO3  Year  \
1092         Togo          TGO  2024
1093         Togo          TGO  2023
1094         Togo          TGO  2022
1095         Togo          TGO  2021
1096         Togo          TGO  2020
…            …           …    …
1149         Togo          TGO  1967
1150         Togo          TGO  1966
1151         Togo          TGO  1965
1152         Togo          TGO  1964
1153         Togo          TGO  1963

                                       Indicator Name  Indicator Code  \
1092  Agriculture, forestry, and fishing, value adde…  NV.AGR.TOTL.ZS
1093  Agriculture, forestry, and fishing, value adde…  NV.AGR.TOTL.ZS
1094  Agriculture, forestry, and fishing, value adde…  NV.AGR.TOTL.ZS
1095  Agriculture, forestry, and fishing, value adde…  NV.AGR.TOTL.ZS
1096  Agriculture, forestry, and fishing, value adde…  NV.AGR.TOTL.ZS
```

```
…                                                    …              …
1149  Agriculture, forestry, and fishing, value adde…  NV.AGR.TOTL.ZS
1150  Agriculture, forestry, and fishing, value adde…  NV.AGR.TOTL.ZS
1151  Agriculture, forestry, and fishing, value adde…  NV.AGR.TOTL.ZS
1152  Agriculture, forestry, and fishing, value adde…  NV.AGR.TOTL.ZS
1153  Agriculture, forestry, and fishing, value adde…  NV.AGR.TOTL.ZS

                 Value
1092   18.039166751545
1093  18.1308320495559
1094  18.7001426667896
1095  18.3649911479173
1096  19.0768869585632
…                  …
1149   44.210528069783
1150  43.8794721278832
1151  47.1264386210453
1152  51.8228906257078
1153  56.1728222232541

[62 rows x 6 columns]
```

[283]:
```python
#convertir la colonne Value en numérique
df_togo["Value"] = pd.to_numeric(df_togo["Value"], errors="coerce")
```

[284]:
```python
#afficher les indicateurs disponibles dans le dataset togo
print("Indicateurs disponibles :", df_togo["Indicator Name"].unique())
```

```
Indicateurs disponibles : ['#indicator+name' 'Agricultural land (sq. km)'
 'Agricultural land (% of land area)' 'Arable land (% of land area)'
 'Rural land area where elevation is below 5 meters (sq. km)'
 'Rural land area where elevation is below 5 meters (% of total land area)'
 'Urban land area where elevation is below 5 meters (sq. km)'
 'Urban land area where elevation is below 5 meters (% of total land area)'
 'Land area where elevation is below 5 meters (% of total land area)'
 'Forest area (sq. km)' 'Forest area (% of land area)'
 'Average precipitation in depth (mm per year)'
 'Cereal yield (kg per hectare)'
 'Foreign direct investment, net inflows (% of GDP)'
 'Access to electricity (% of population)'
 'Electricity production from coal sources (% of total)'
 'Electricity production from hydroelectric sources (% of total)'
 'Electricity production from natural gas sources (% of total)'
 'Electricity production from nuclear sources (% of total)'
 'Electricity production from oil sources (% of total)'
 'Renewable electricity output (% of total electricity output)'
 'Electricity production from renewable sources, excluding hydroelectric (kWh)'
 'Electricity production from renewable sources, excluding hydroelectric (% of
```

```
total)'
 'Renewable energy consumption (% of total final energy consumption)'
 'Energy use (kg of oil equivalent) per $1,000 GDP (constant 2021 PPP)'
 'Electric power consumption (kWh per capita)'
 'Energy use (kg of oil equivalent per capita)'
 'Droughts, floods, extreme temperatures (% of population, average 1990-2009)'
 'Rural population living in areas where elevation is below 5 meters (% of total
population)'
 'Urban population living in areas where elevation is below 5 meters (% of total
population)'
 'Population living in areas where elevation is below 5 meters (% of total
population)'
 'Population in urban agglomerations of more than 1 million (% of total
population)'
 'Annual freshwater withdrawals, total (billion cubic meters)'
 'Annual freshwater withdrawals, total (% of internal resources)'
 'Terrestrial protected areas (% of total land area)'
 'Marine protected areas (% of territorial waters)'
 'Terrestrial and marine protected areas (% of total territorial area)'
 'CPIA public sector management and institutions cluster average (1=low to
6=high)'
 'Agriculture, forestry, and fishing, value added (% of GDP)'
 'School enrollment, primary and secondary (gross), gender parity index (GPI)'
 'Primary completion rate, total (% of relevant age group)'
 'Mortality rate, under-5 (per 1,000 live births)'
 'Prevalence of underweight, weight for age (% of children under 5)'
 'Poverty headcount ratio at $3.00 a day (2021 PPP) (% of population)'
 'Population growth (annual %)' 'Population, total'
 'Urban population growth (annual %)' 'Urban population'
 'Urban population (% of total population)'
 'Roads, paved (% of total roads)']
```

7 A. AXE 1 _ PERFORMANCE ECONOMIQUE

[285]:
```python
# 1 Filtrage des indicateurs pertinents
indicateurs = [
    "Population, total",
    "Agricultural land (sq. km)",
    "Agriculture, forestry, and fishing, value added (% of GDP)",
    "Foreign direct investment, net inflows (% of GDP)",
    "Energy use (kg of oil equivalent per capita)",
    "Access to electricity (% of population)",
    "Average precipitation in depth (mm per year)",
    "Forest area (% of land area)"
]

# 2 Filtrage du DataFrame
```

```python
togo_filtered = df_togo[df_togo["Indicator Name"].isin(indicateurs)].copy()

# 3 Conversion des valeurs en numériques
togo_filtered["Value"] = pd.to_numeric(togo_filtered["Value"], errors="coerce")

# 4 Transformation du dataset pour avoir une colonne par indicateur
togo_pivot = togo_filtered.pivot(index="Year", columns="Indicator Name",␣
↪values="Value").reset_index()

# 5 Nettoyage des noms de colonnes
togo_pivot.columns = [col.strip() for col in togo_pivot.columns]

# 6 Conversion de l'année en entier
togo_pivot["Year"] = pd.to_numeric(togo_pivot["Year"], errors="coerce").
↪astype("Int64")

# 7 Tri chronologique
togo_pivot = togo_pivot.sort_values("Year").reset_index(drop=True)

# 8 Création de variables dérivées
# Surface agricole par habitant
togo_pivot["Agricultural land per capita (sq. km)"] = togo_pivot["Agricultural␣
↪land (sq. km)"] / togo_pivot["Population, total"]

# Ratio énergie par hectare agricole
togo_pivot["Energy use per agricultural km²"] = togo_pivot["Energy use (kg of␣
↪oil equivalent per capita)"] / togo_pivot["Agricultural land per capita (sq.␣
↪km)"]

# 9 Aperçu du dataset enrichi
print("Aperçu du dataset structuré et enrichi :")
print(togo_pivot.head())
```

```
Aperçu du dataset structuré et enrichi :
   Year  Access to electricity (% of population)  Agricultural land (sq. km)  \
0  1960                                       NaN                         NaN
1  1961                                       NaN                     30700.0
2  1962                                       NaN                     30700.0
3  1963                                       NaN                     30700.0
4  1964                                       NaN                     30700.0

   Agriculture, forestry, and fishing, value added (% of GDP)  \
0                                                        NaN
1                                                        NaN
2                                                        NaN
3                                                  56.172822
4                                                  51.822891
```

```
   Average precipitation in depth (mm per year)  \
0                                            NaN
1                                         1168.0
2                                         1168.0
3                                         1168.0
4                                         1168.0

   Energy use (kg of oil equivalent per capita)  \
0                                            NaN
1                                            NaN
2                                            NaN
3                                            NaN
4                                            NaN

   Foreign direct investment, net inflows (% of GDP)  \
0                                                NaN
1                                                NaN
2                                                NaN
3                                                NaN
4                                                NaN

   Forest area (% of land area)  Population, total  \
0                           NaN          1673110.0
1                           NaN          1702717.0
2                           NaN          1732959.0
3                           NaN          1765072.0
4                           NaN          1803440.0

   Agricultural land per capita (sq. km)  Energy use per agricultural km²
0                                    NaN                               NaN
1                               0.018030                               NaN
2                               0.017715                               NaN
3                               0.017393                               NaN
4                               0.017023                               NaN
```

[286]:
```
# Transformation du dataset CO2 en format long et nettoyage des types
# Étape 1 : Supprimer la colonne inutile
df_c02 = df_c02.drop(columns=["Unnamed: 69"], errors="ignore")

# Étape 2 : Transformation en format long
df_co2_long = df_c02.melt(
    id_vars=["Country Name", "Country Code", "Indicator Name", "Indicator␣
 ↪Code"],
    var_name="Year",
    value_name="CO2 Emissions"
)
```

```
# Étape 3 : Nettoyage des types
df_co2_long["Year"] = df_co2_long["Year"].astype(str).str.extract(r"(\d{4})") ␣
↪# garde uniquement les années valides
df_co2_long["Year"] = pd.to_numeric(df_co2_long["Year"], errors="coerce").
↪astype("Int64")
df_co2_long["CO2 Emissions"] = pd.to_numeric(df_co2_long["CO2 Emissions"],␣
↪errors="coerce")
```

[287]:
```
# Aperçu du dataset transformé
print(" Années CO  disponibles :", sorted(df_co2_long["Year"].dropna().
↪unique()))
```

```
 Années CO  disponibles : [np.int64(1960), np.int64(1961), np.int64(1962),
np.int64(1963), np.int64(1964), np.int64(1965), np.int64(1966), np.int64(1967),
np.int64(1968), np.int64(1969), np.int64(1970), np.int64(1971), np.int64(1972),
np.int64(1973), np.int64(1974), np.int64(1975), np.int64(1976), np.int64(1977),
np.int64(1978), np.int64(1979), np.int64(1980), np.int64(1981), np.int64(1982),
np.int64(1983), np.int64(1984), np.int64(1985), np.int64(1986), np.int64(1987),
np.int64(1988), np.int64(1989), np.int64(1990), np.int64(1991), np.int64(1992),
np.int64(1993), np.int64(1994), np.int64(1995), np.int64(1996), np.int64(1997),
np.int64(1998), np.int64(1999), np.int64(2000), np.int64(2001), np.int64(2002),
np.int64(2003), np.int64(2004), np.int64(2005), np.int64(2006), np.int64(2007),
np.int64(2008), np.int64(2009), np.int64(2010), np.int64(2011), np.int64(2012),
np.int64(2013), np.int64(2014), np.int64(2015), np.int64(2016), np.int64(2017),
np.int64(2018), np.int64(2019), np.int64(2020), np.int64(2021), np.int64(2022),
np.int64(2023), np.int64(2024)]
```

[288]:
```
print(df_co2_long.columns)
```

```
Index(['Country Name', 'Country Code', 'Indicator Name', 'Indicator Code',
       'Year', 'CO2 Emissions'],
      dtype='object')
```

[289]:
```
#  Afficher les pays disponibles
print("Pays disponibles :", df_co2_long["Country Name"].unique())
```

```
Pays disponibles : ['Aruba' 'Africa Eastern and Southern' 'Afghanistan'
 'Africa Western and Central' 'Angola' 'Albania' 'Andorra' 'Arab World'
 'United Arab Emirates' 'Argentina' 'Armenia' 'American Samoa'
 'Antigua and Barbuda' 'Australia' 'Austria' 'Azerbaijan' 'Burundi'
 'Belgium' 'Benin' 'Burkina Faso' 'Bangladesh' 'Bulgaria' 'Bahrain'
 'Bahamas, The' 'Bosnia and Herzegovina' 'Belarus' 'Belize' 'Bermuda'
 'Bolivia' 'Brazil' 'Barbados' 'Brunei Darussalam' 'Bhutan' 'Botswana'
 'Central African Republic' 'Canada' 'Central Europe and the Baltics'
 'Switzerland' 'Channel Islands' 'Chile' 'China' "Cote d'Ivoire"
 'Cameroon' 'Congo, Dem. Rep.' 'Congo, Rep.' 'Colombia' 'Comoros'
 'Cabo Verde' 'Costa Rica' 'Caribbean small states' 'Cuba' 'Curacao'
 'Cayman Islands' 'Cyprus' 'Czechia' 'Germany' 'Djibouti' 'Dominica'
```

'Denmark' 'Dominican Republic' 'Algeria'
'East Asia & Pacific (excluding high income)'
'Early-demographic dividend' 'East Asia & Pacific'
'Europe & Central Asia (excluding high income)' 'Europe & Central Asia'
'Ecuador' 'Egypt, Arab Rep.' 'Euro area' 'Eritrea' 'Spain' 'Estonia'
'Ethiopia' 'European Union' 'Fragile and conflict affected situations'
'Finland' 'Fiji' 'France' 'Faroe Islands' 'Micronesia, Fed. Sts.' 'Gabon'
'United Kingdom' 'Georgia' 'Ghana' 'Gibraltar' 'Guinea' 'Gambia, The'
'Guinea-Bissau' 'Equatorial Guinea' 'Greece' 'Grenada' 'Greenland'
'Guatemala' 'Guam' 'Guyana' 'High income' 'Hong Kong SAR, China'
'Honduras' 'Heavily indebted poor countries (HIPC)' 'Croatia' 'Haiti'
'Hungary' 'IBRD only' 'IDA & IBRD total' 'IDA total' 'IDA blend'
'Indonesia' 'IDA only' 'Isle of Man' 'India' 'Not classified' 'Ireland'
'Iran, Islamic Rep.' 'Iraq' 'Iceland' 'Israel' 'Italy' 'Jamaica' 'Jordan'
'Japan' 'Kazakhstan' 'Kenya' 'Kyrgyz Republic' 'Cambodia' 'Kiribati'
'St. Kitts and Nevis' 'Korea, Rep.' 'Kuwait'
'Latin America & Caribbean (excluding high income)' 'Lao PDR' 'Lebanon'
'Liberia' 'Libya' 'St. Lucia' 'Latin America & Caribbean'
'Least developed countries: UN classification' 'Low income'
'Liechtenstein' 'Sri Lanka' 'Lower middle income' 'Low & middle income'
'Lesotho' 'Late-demographic dividend' 'Lithuania' 'Luxembourg' 'Latvia'
'Macao SAR, China' 'St. Martin (French part)' 'Morocco' 'Monaco'
'Moldova' 'Madagascar' 'Maldives'
'Middle East, North Africa, Afghanistan & Pakistan' 'Mexico'
'Marshall Islands' 'Middle income' 'North Macedonia' 'Mali' 'Malta'
'Myanmar'
'Middle East, North Africa, Afghanistan & Pakistan (excluding high income)'
'Montenegro' 'Mongolia' 'Northern Mariana Islands' 'Mozambique'
'Mauritania' 'Mauritius' 'Malawi' 'Malaysia' 'North America' 'Namibia'
'New Caledonia' 'Niger' 'Nigeria' 'Nicaragua' 'Netherlands' 'Norway'
'Nepal' 'Nauru' 'New Zealand' 'OECD members' 'Oman' 'Other small states'
'Pakistan' 'Panama' 'Peru' 'Philippines' 'Palau' 'Papua New Guinea'
'Poland' 'Pre-demographic dividend' 'Puerto Rico (US)'
"Korea, Dem. People's Rep." 'Portugal' 'Paraguay' 'West Bank and Gaza'
'Pacific island small states' 'Post-demographic dividend'
'French Polynesia' 'Qatar' 'Romania' 'Russian Federation' 'Rwanda'
'South Asia' 'Saudi Arabia' 'Sudan' 'Senegal' 'Singapore'
'Solomon Islands' 'Sierra Leone' 'El Salvador' 'San Marino'
'Somalia, Fed. Rep.' 'Serbia'
'Sub-Saharan Africa (excluding high income)' 'South Sudan'
'Sub-Saharan Africa' 'Small states' 'Sao Tome and Principe' 'Suriname'
'Slovak Republic' 'Slovenia' 'Sweden' 'Eswatini'
'Sint Maarten (Dutch part)' 'Seychelles' 'Syrian Arab Republic'
'Turks and Caicos Islands' 'Chad'
'East Asia & Pacific (IDA & IBRD countries)'
'Europe & Central Asia (IDA & IBRD countries)' 'Togo' 'Thailand'
'Tajikistan' 'Turkmenistan'
'Latin America & the Caribbean (IDA & IBRD countries)' 'Timor-Leste'

```
'Middle East, North Africa, Afghanistan & Pakistan (IDA & IBRD)' 'Tonga'
'South Asia (IDA & IBRD)' 'Sub-Saharan Africa (IDA & IBRD countries)'
'Trinidad and Tobago' 'Tunisia' 'Turkiye' 'Tuvalu' 'Tanzania' 'Uganda'
'Ukraine' 'Upper middle income' 'Uruguay' 'United States' 'Uzbekistan'
'St. Vincent and the Grenadines' 'Venezuela, RB' 'British Virgin Islands'
'Virgin Islands (U.S.)' 'Viet Nam' 'Vanuatu' 'World' 'Samoa' 'Kosovo'
'Yemen, Rep.' 'South Africa' 'Zambia' 'Zimbabwe']
```

[290]:
```
# Afficher les indicateurs disponibles pour le Togo (facultatif pour␣
↪vérification)
print(df_co2_long[df_co2_long["Country Name"] == "Togo"]["Indicator Name"].
↪unique())

# Filtrer les données CO  par habitant pour le Togo
co2_togo = df_co2_long[
    (df_co2_long["Country Name"] == "Togo") &
    (df_co2_long["Indicator Name"] == "Carbon dioxide (CO2) emissions excluding␣
↪LULUCF per capita (t CO2e/capita)")
].copy()

# Nettoyage
# Nettoyage
co2_togo["Year"] = pd.to_numeric(co2_togo["Year"], errors="coerce").
↪astype("Int64")
co2_togo["CO2_per_capita"] = pd.to_numeric(co2_togo["CO2 Emissions"],␣
↪errors="coerce")
co2_togo = co2_togo[["Year", "CO2_per_capita"]].dropna()
```

```
['Carbon dioxide (CO2) emissions excluding LULUCF per capita (t CO2e/capita)']
```

7.0.1 hypothèse : Quand l'économie du Togo se développe, la consommation d'énergie augmente aussi, surtout avec des sources non renouvelables. Cela provoque une hausse des émissions de CO par personne.

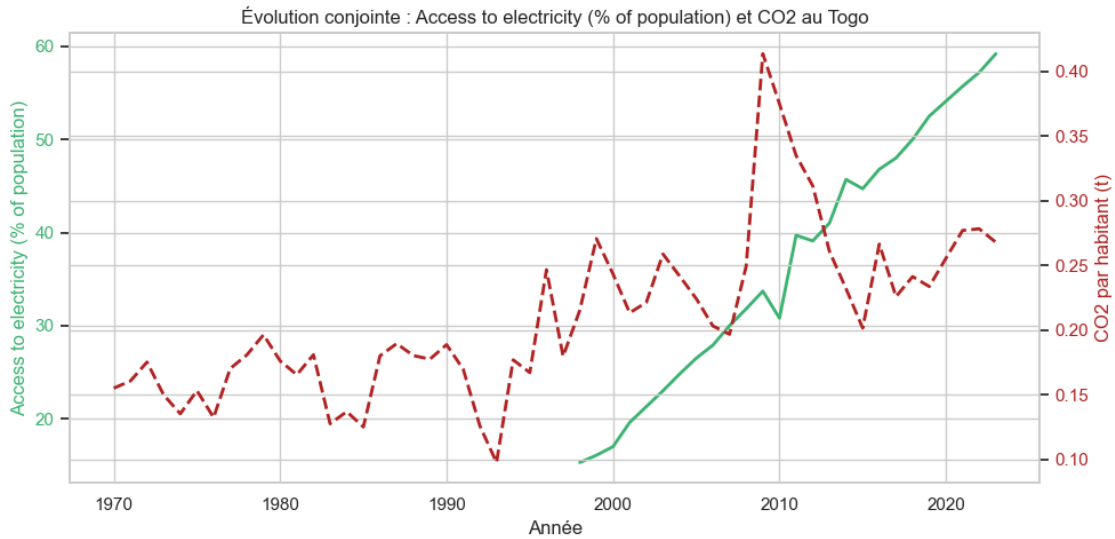
[291]:
```
# Liste complète des indicateurs économiques à analyser
indicateurs_economiques = [
    "Foreign direct investment, net inflows (% of GDP)",
    "Agriculture, forestry, and fishing, value added (% of GDP)",
    "Access to electricity (% of population)",
    "Energy use (kg of oil equivalent per capita)",
    "Electric power consumption (kWh per capita)",
    "Renewable energy consumption (% of total final energy consumption)",
    "Electricity production from coal sources (% of total)",
    "Electricity production from hydroelectric sources (% of total)",
    "Electricity production from oil sources (% of total)",
    "Electricity production from renewable sources, excluding hydroelectric (%␣
↪of total)"
```

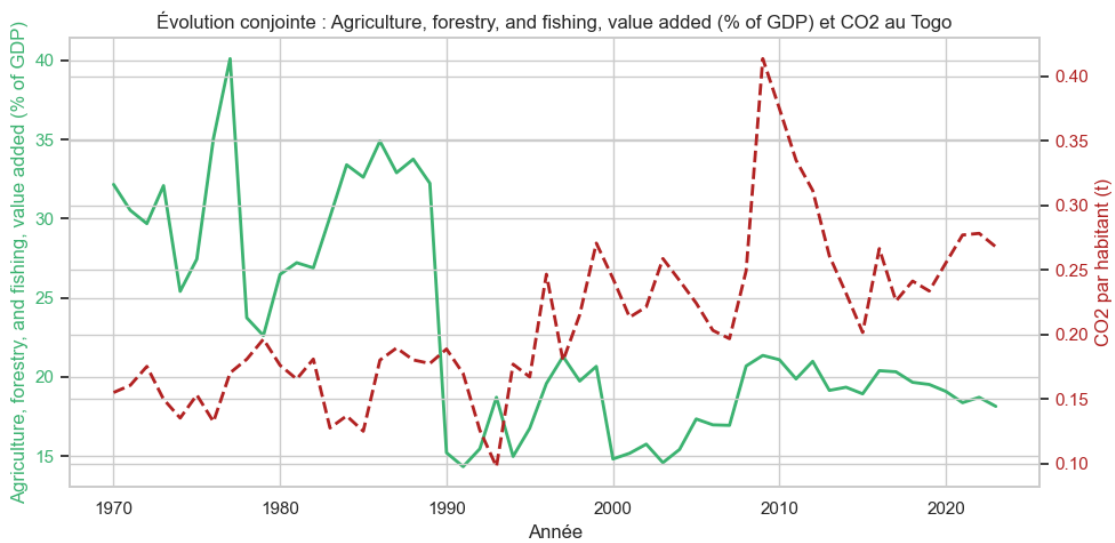
```
]

# Filtrer les données du Togo avec les indicateurs choisis
economie_togo = df_togo[df_togo["Indicator Name"].
 ↪isin(indicateurs_economiques)].copy()

# Nettoyage
economie_togo["Value"] = pd.to_numeric(economie_togo["Value"], errors="coerce")
economie_togo["Year"] = pd.to_numeric(economie_togo["Year"], errors="coerce").
 ↪astype("Int64")

# Transformation en format pivot (lignes = années, colonnes = indicateurs)
economie_pivot = economie_togo.pivot_table(
    index="Year",
    columns="Indicator Name",
    values="Value"
).reset_index()
```

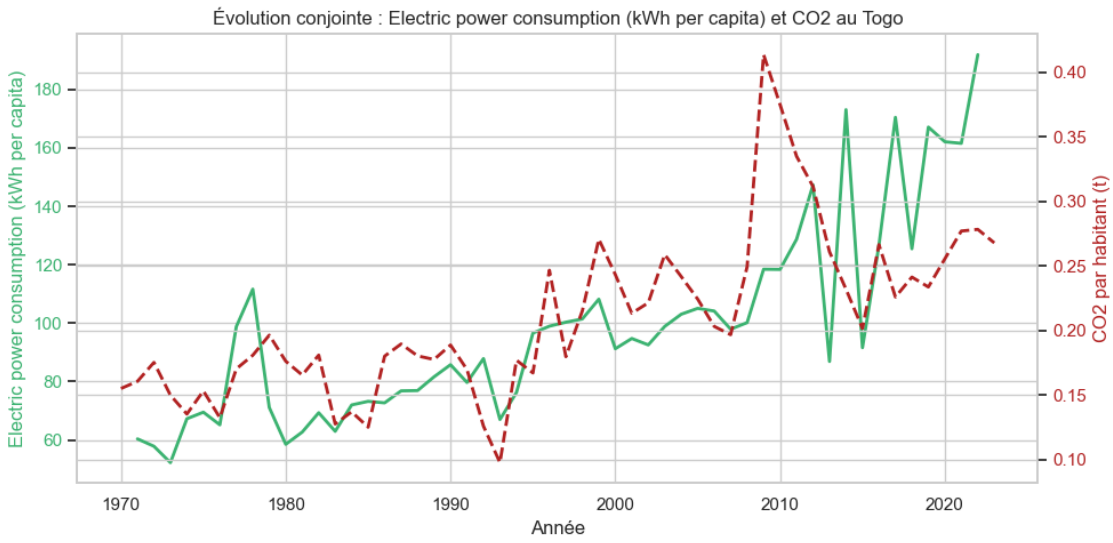
[292]:
```
# Fusion sur l'année
merged = pd.merge(
    economie_pivot,
    co2_togo,
    on="Year",
    how="inner"
)
```

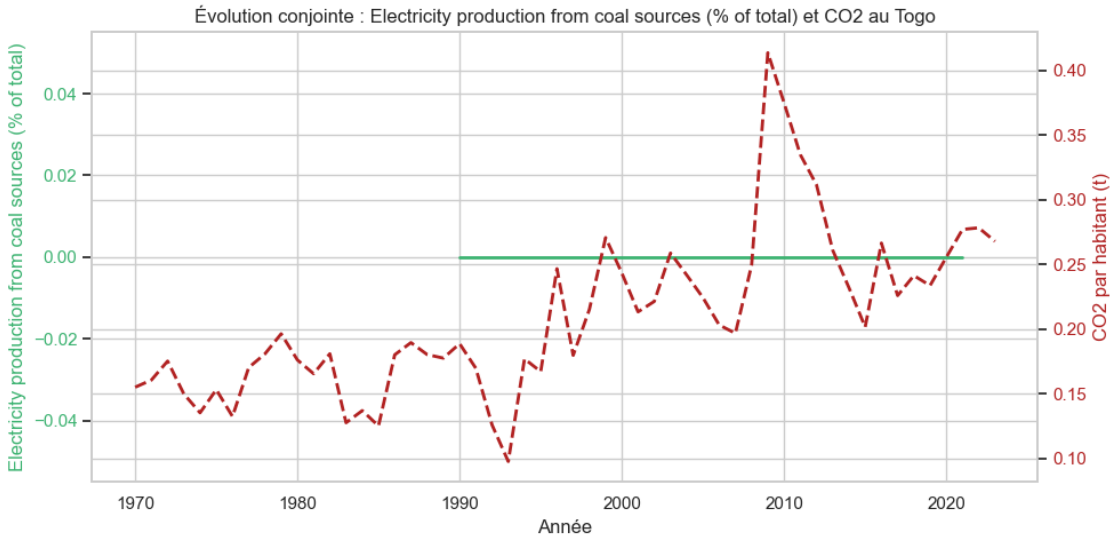
[293]:
```
# Liste des indicateurs économiques
indicateurs_economiques = [col for col in merged.columns if col not in ["Year",␣
 ↪"CO2_per_capita"]]

# Boucle de visualisation
for indicateur in indicateurs_economiques:
    fig, ax1 = plt.subplots(figsize=(10, 5))

    # Axe Y gauche : indicateur économique
    ax1.plot(merged["Year"], merged[indicateur], color='mediumseagreen',␣
 ↪linewidth=2)
    ax1.set_ylabel(indicateur, color='mediumseagreen')
    ax1.tick_params(axis='y', labelcolor='mediumseagreen')

    # Axe Y droit : CO2 par habitant
    ax2 = ax1.twinx()
    ax2.plot(merged["Year"], merged["CO2_per_capita"], color='firebrick',␣
 ↪linestyle='--', linewidth=2)
    ax2.set_ylabel("CO2 par habitant (t)", color='firebrick')
    ax2.tick_params(axis='y', labelcolor='firebrick')
```

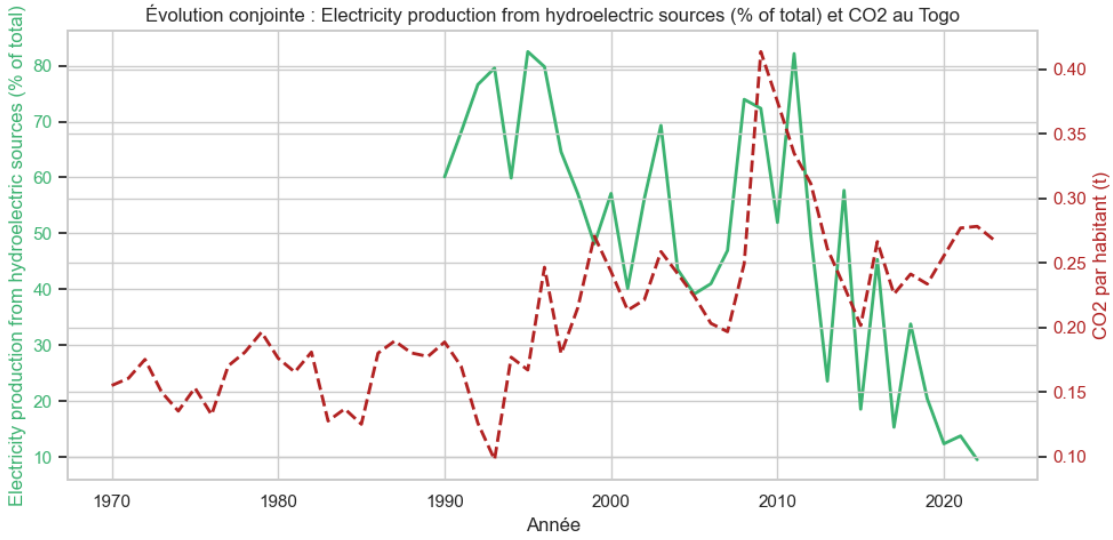
```
    plt.title(f"Évolution conjointe : {indicateur} et CO2 au Togo")
    ax1.set_xlabel("Année")
    fig.tight_layout()
    plt.grid(True)
    plt.show()
```

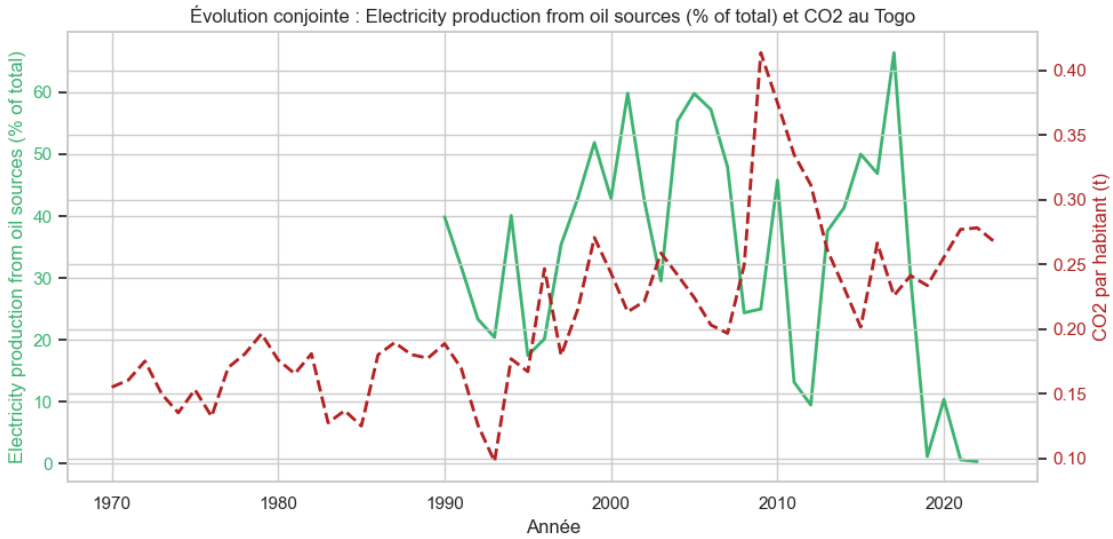


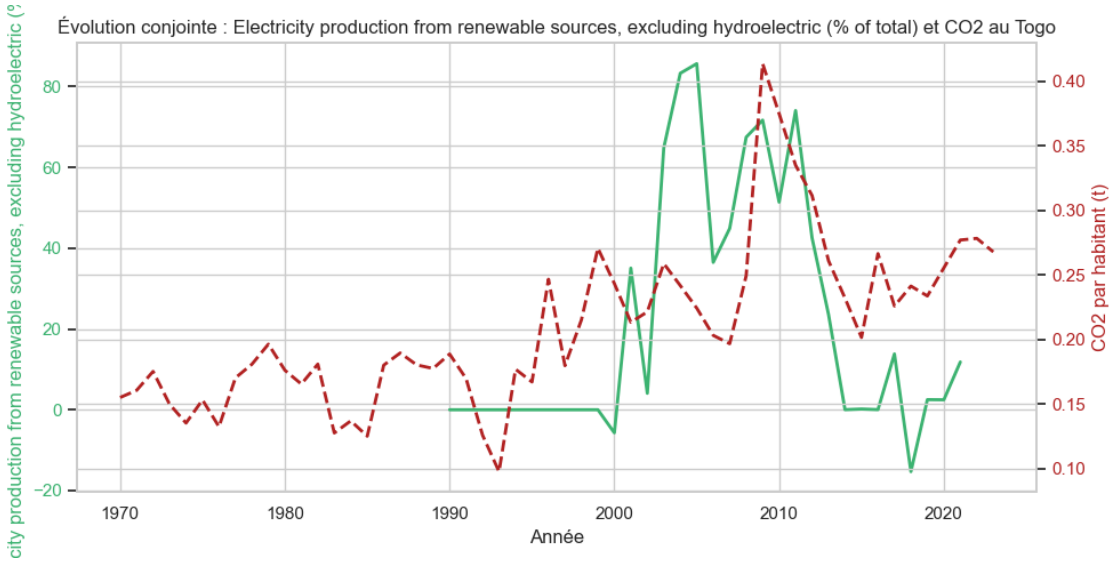


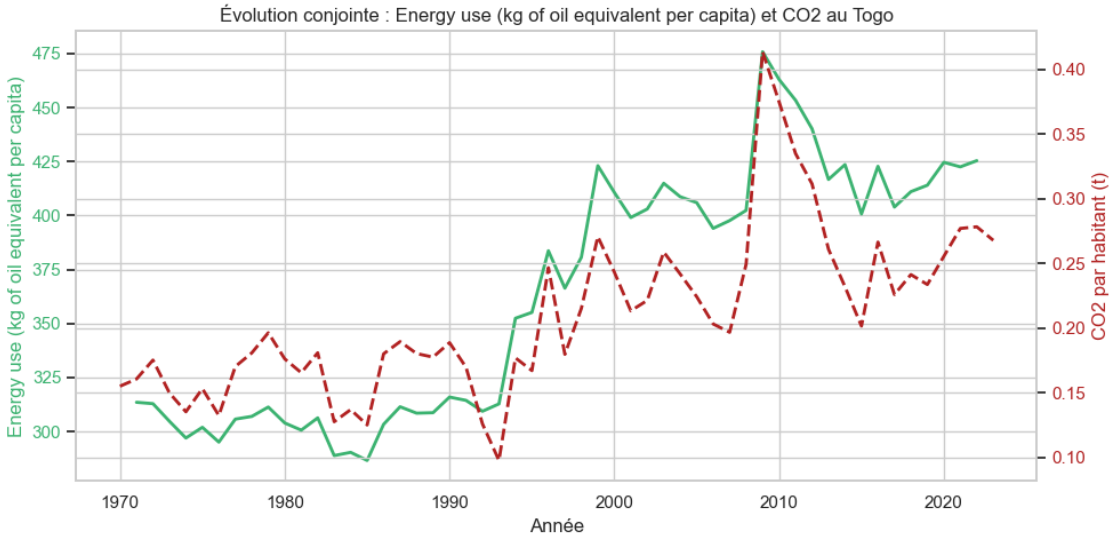

Évolution conjointe : Electric power consumption (kWh per capita) et CO2 au Togo
Electric power consumption (kWh per capita)
CO2 par habitant (t)
Année
180
160
140
120
100
80
60
0.40
0.35
0.30
0.25
0.20
0.15
0.10
1970
1980
1990
2000
2010
2020

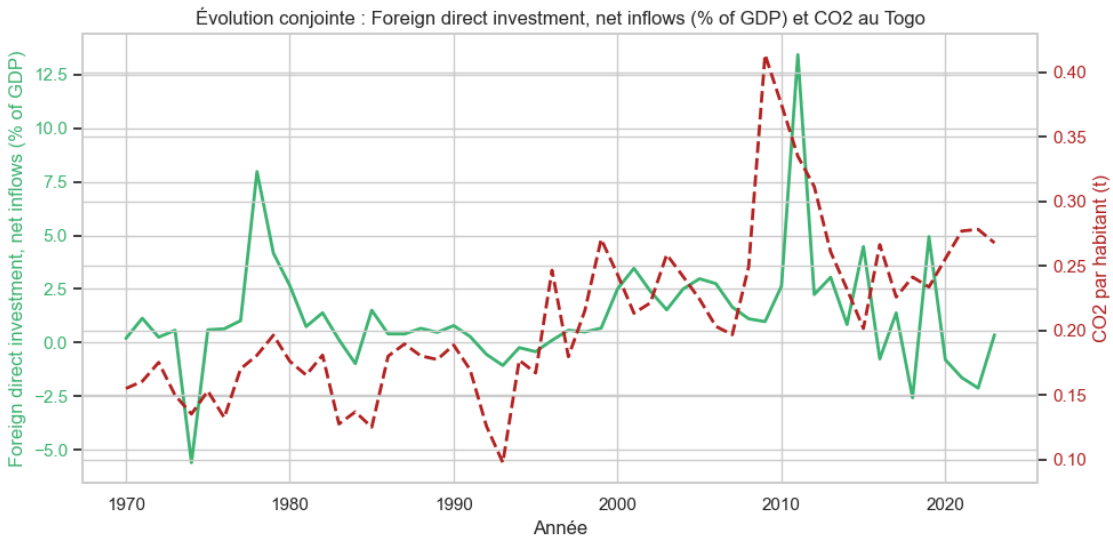

Évolution conjointe : Electricity production from coal sources (% of total) et CO2 au Togo
Electricity production from coal sources (% of total)
CO2 par habitant (t)
Année
0.04
0.02
0.00
−0.02
−0.04
0.40
0.35
0.30
0.25
0.20
0.15
0.10
1970
1980
1990
2000
2010
2020

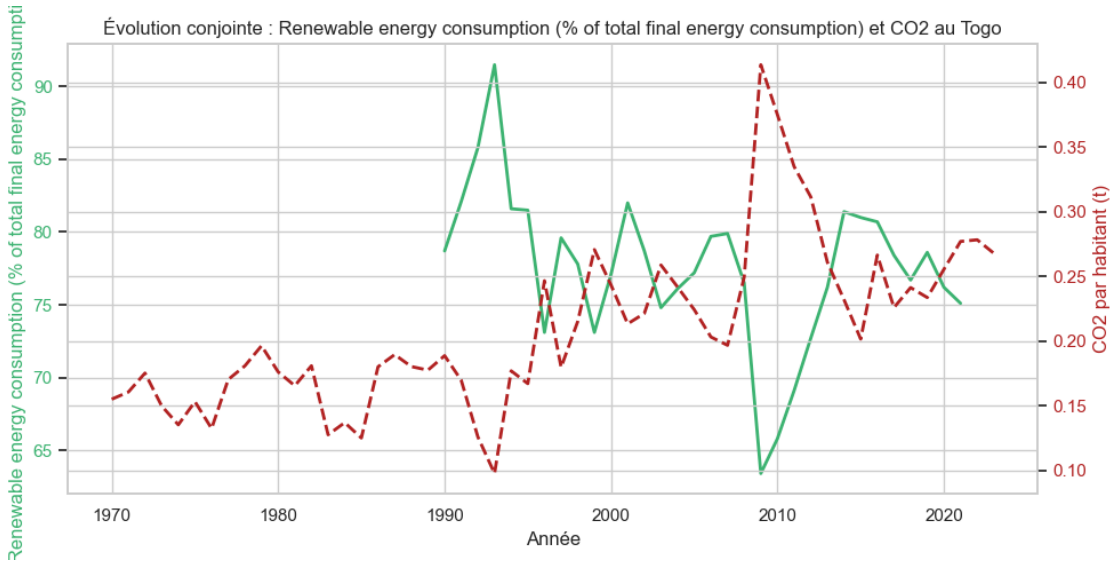

Évolution conjointe : Electricity production from hydroelectric sources (% of total) et CO2 au Togo
Electricity production from hydroelectric sources (% of total)
CO2 par habitant (t)
Année
10
20
30
40
50
60
70
80
0.10
0.15
0.20
0.25
0.30
0.35
0.40
1970
1980
1990
2000
2010
2020


Évolution conjointe : Electricity production from oil sources (% of total) et CO2 au Togo
Electricity production from oil sources (% of total)
CO2 par habitant (t)
Année
0
10
20
30
40
50
60
0.10
0.15
0.20
0.25
0.30
0.35
0.40
1970
1980
1990
2000
2010
2020


Évolution conjointe : Electricity production from renewable sources, excluding hydroelectric (% of total) et CO2 au Togo
city production from renewable sources, excluding hydroelectric (%
CO2 par habitant (t)
Année
80
60
40
20
0
−20
0.40
0.35
0.30
0.25
0.20
0.15
0.10
1970
1980
1990
2000
2010
2020


Évolution conjointe : Energy use (kg of oil equivalent per capita) et CO2 au Togo
Energy use (kg of oil equivalent per capita)
CO2 par habitant (t)
Année
475
450
425
400
375
350
325
300
0.40
0.35
0.30
0.25
0.20
0.15
0.10
1970
1980
1990
2000
2010
2020





7.0.2 Agriculture (% PIB) vs CO

Observation

Agriculture part GDP ↓ chute forte après 1990

CO ↑ peu (activité économique faible)

Explication

Togo diversifie économie → services

Crises 1990 et 2005 → baisse production

Dates clés 1990–1993 crise → agriculture effondre temporairement 2005 crise → même effet

[294]:
```
# Calcul et affichage des corrélations
for indicateur in indicateurs_economiques:
    corr = merged["CO2_per_capita"].corr(merged[indicateur])
    print(f"Corrélation entre CO  par habitant et {indicateur} : {corr:.3f}")
```

```
Corrélation entre CO  par habitant et Access to electricity (% of population) :
0.129
Corrélation entre CO  par habitant et Agriculture, forestry, and fishing, value
added (% of GDP) : -0.426
Corrélation entre CO  par habitant et Electric power consumption (kWh per
capita) : 0.634
Corrélation entre CO  par habitant et Electricity production from coal sources
(% of total) : nan
Corrélation entre CO  par habitant et Electricity production from hydroelectric
sources (% of total) : -0.150
Corrélation entre CO  par habitant et Electricity production from oil sources (%
of total) : -0.160
Corrélation entre CO  par habitant et Electricity production from renewable
sources, excluding hydroelectric (% of total) : 0.497
Corrélation entre CO  par habitant et Energy use (kg of oil equivalent per
capita) : 0.887
Corrélation entre CO  par habitant et Foreign direct investment, net inflows (%
of GDP) : 0.293
Corrélation entre CO  par habitant et Renewable energy consumption (% of total
final energy consumption) : -0.941
```

```
c:\Users\fangn\OneDrive\Bureau\BACHELOR 2 NEXA DIGITAL SCHOOL\SEMESTRE
1\EVOLUTION DU CLIMAT  CERVELLO\env\Lib\site-
packages\numpy\lib\_function_base_impl.py:3065: RuntimeWarning: invalid value
encountered in divide
  c /= stddev[:, None]
c:\Users\fangn\OneDrive\Bureau\BACHELOR 2 NEXA DIGITAL SCHOOL\SEMESTRE
1\EVOLUTION DU CLIMAT  CERVELLO\env\Lib\site-
packages\numpy\lib\_function_base_impl.py:3066: RuntimeWarning: invalid value
encountered in divide
  c /= stddev[None, :]
```

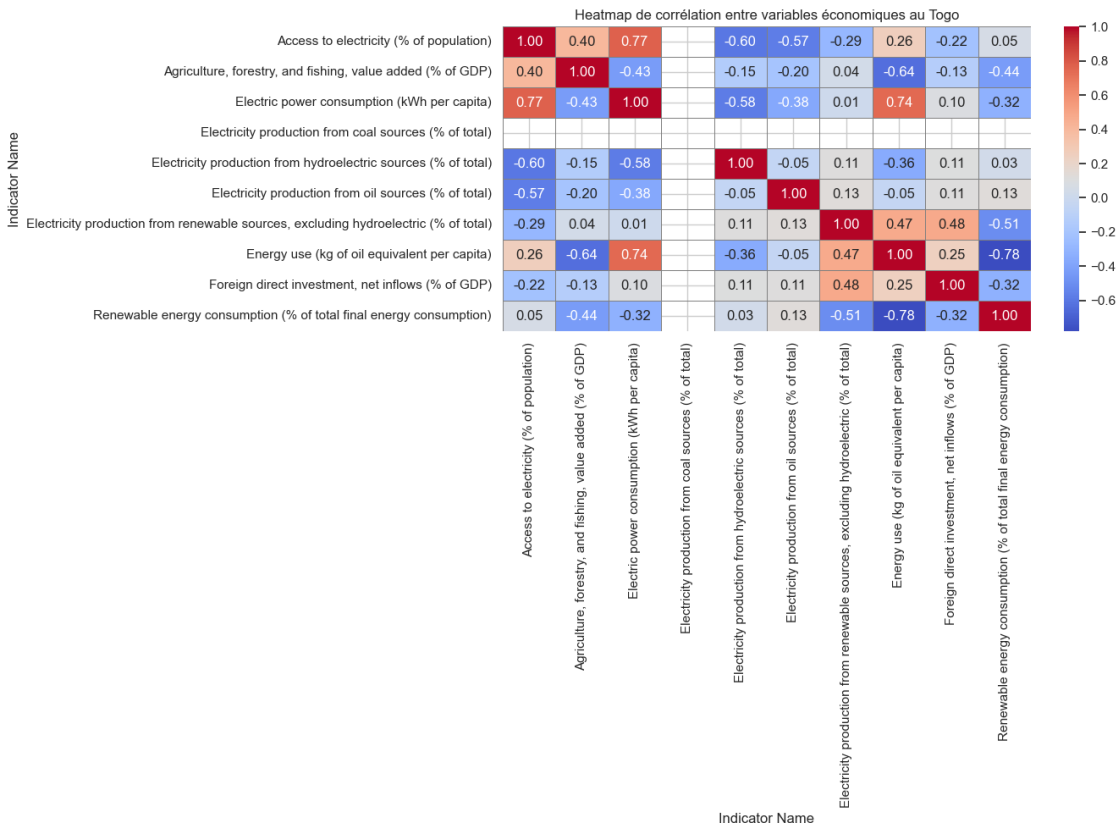
Energy use (kg of oil equivalent per capita) +0.887 Très forte corrélation : plus l'économie consomme d'énergie, plus les émissions augmentent. Electric power consumption (kWh per capita) +0.634 Forte corrélation : la consommation électrique croît avec les émissions, signe d'un développement énergivore. Foreign direct investment (% of GDP) +0.293 Corrélation modérée : les investissements étrangers accompagnent la croissance et l'intensification énergétique. Access to electricity (% of population) +0.129 Corrélation faible mais positive : l'électrification progresse avec le développement, contribuant aux émissions.

Les données suggèrent que la croissance économique du Togo est actuellement couplée à une hausse de la consommation énergétique, en grande partie issue de sources non renouvelables, ce qui entraîne une augmentation des émissions de CO par habitant. La transition énergétique, notamment via

les renouvelables, apparaît comme un levier stratégique pour réduire l'intensité carbone de cette croissance.

[295]:
```
#visualisation de la heatmap de corrélation entre plusieurs variables␣
↪économiques au Togo


#  Étape 1 : Définir les indicateurs économiques à analyser
indicateurs_economiques = [
    "Foreign direct investment, net inflows (% of GDP)",
    "Agriculture, forestry, and fishing, value added (% of GDP)",
    "Access to electricity (% of population)",
    "Energy use (kg of oil equivalent per capita)",
    "Electric power consumption (kWh per capita)",
    "Renewable energy consumption (% of total final energy consumption)",
    "Electricity production from coal sources (% of total)",
    "Electricity production from hydroelectric sources (% of total)",
    "Electricity production from oil sources (% of total)",
    "Electricity production from renewable sources, excluding hydroelectric (%␣
↪of total)"
]

#  Étape 2 : Filtrer les données du Togo avec les indicateurs choisis
df_econ = df_togo[df_togo["Indicator Name"].isin(indicateurs_economiques)].
↪copy()

# Nettoyage des colonnes
df_econ["Year"] = pd.to_numeric(df_econ["Year"], errors="coerce").
↪astype("Int64")
df_econ["Value"] = pd.to_numeric(df_econ["Value"], errors="coerce")

# Étape 3 : Transformer en format pivot (lignes = années, colonnes =␣
↪indicateurs)
df_econ_pivot = df_econ.pivot_table(
    index="Year",
    columns="Indicator Name",
    values="Value"
)

#  Étape 4 : Calculer la matrice de corrélation
corr_matrix = df_econ_pivot.corr()

# Étape 5 : Visualiser la heatmap avec seaborn
plt.figure(figsize=(14, 10))
sns.heatmap(
    corr_matrix,
    annot=True,            # Affiche les valeurs numériques
```

```
    fmt=".2f",              # Format des décimales
    cmap="coolwarm",        # Palette de couleurs
    linewidths=0.5,         # Séparation entre les cases
    linecolor='gray'        # Couleur des lignes
)

plt.title("Heatmap de corrélation entre variables économiques au Togo")
plt.tight_layout()
plt.show()
```



7.0.3 Hypothèse : Au Togo, quand les températures augmentent, l'économie change : on utilise plus d'énergie, l'agriculture devient moins importante, et le pays attire plus d'investissements étrangers.

[296]:
```
# Liste des indicateurs économiques à visualiser
indicateurs = [
    "Foreign direct investment, net inflows (% of GDP)",
    "Agriculture, forestry, and fishing, value added (% of GDP)",
    "Energy use (kg of oil equivalent per capita)",
    "Access to electricity (% of population)",
    "Electric power consumption (kWh per capita)"
```

```
]

#  Préparation des données de température pour le Togo
df_temp_togo = df_temp_count[df_temp_count["Country"] == "Togo"].copy()
df_temp_togo["Year"] = pd.to_datetime(df_temp_togo["dt"]).dt.year
df_temp_annual = df_temp_togo.groupby("Year")["AverageTemperature"].mean().
 ↪reset_index()

#  Préparation des données économiques
df_econ = df_togo[df_togo["Indicator Name"].isin(indicateurs)].copy()
df_econ["Year"] = pd.to_numeric(df_econ["Year"], errors="coerce").
 ↪astype("Int64")
df_econ["Value"] = pd.to_numeric(df_econ["Value"], errors="coerce")

# Pivot pour avoir une colonne par indicateur
df_econ_pivot = df_econ.pivot_table(
    index="Year",
    columns="Indicator Name",
    values="Value"
).reset_index()

# Fusion température + économie
df_merged = pd.merge(df_temp_annual, df_econ_pivot, on="Year", how="inner")
```

[297]:
```
# Calcul et affichage des corrélations
for indicateur in indicateurs:
    corr = df_merged["AverageTemperature"].corr(df_merged[indicateur])
    print(f" Corrélation entre température et {indicateur} : {corr:.3f}")
```

```
 Corrélation entre température et Foreign direct investment, net inflows (% of
GDP) : 0.313
 Corrélation entre température et Agriculture, forestry, and fishing, value
added (% of GDP) : -0.531
 Corrélation entre température et Energy use (kg of oil equivalent per capita) :
0.658
 Corrélation entre température et Access to electricity (% of population) :
0.332
 Corrélation entre température et Electric power consumption (kWh per capita) :
0.501
```

Agriculture, forestry and fishing (% of GDP) –0.531 Corrélation négative forte : plus la température augmente, plus la part de l'agriculture dans l'économie diminue. Cela indique une perte de poids des secteurs traditionnels.

Energy use (kg of oil equivalent per capita) +0.658 Corrélation forte : la consommation d'énergie augmente avec la température, ce qui reflète une intensification des activités économiques.

Electric power consumption (kWh per capita) +0.501 Corrélation modérée : la consommation électrique progresse avec la température, signe d'un développement plus énergivore.

Access to electricity (% of population) +0.332 Corrélation faible à modérée : l'accès à l'électricité s'améliore avec la température, ce qui peut refléter une meilleure couverture des infrastructures.

Foreign direct investment, net inflows (% of GDP) +0.313 Corrélation modérée : les investissements étrangers augmentent avec la température, ce qui peut indiquer une attractivité croissante des zones plus développées ou urbanisées.

Ces corrélations montrent que la hausse des températures au Togo est liée à une modernisation économique :

Moins d'agriculture, mais plus d'énergie consommée,

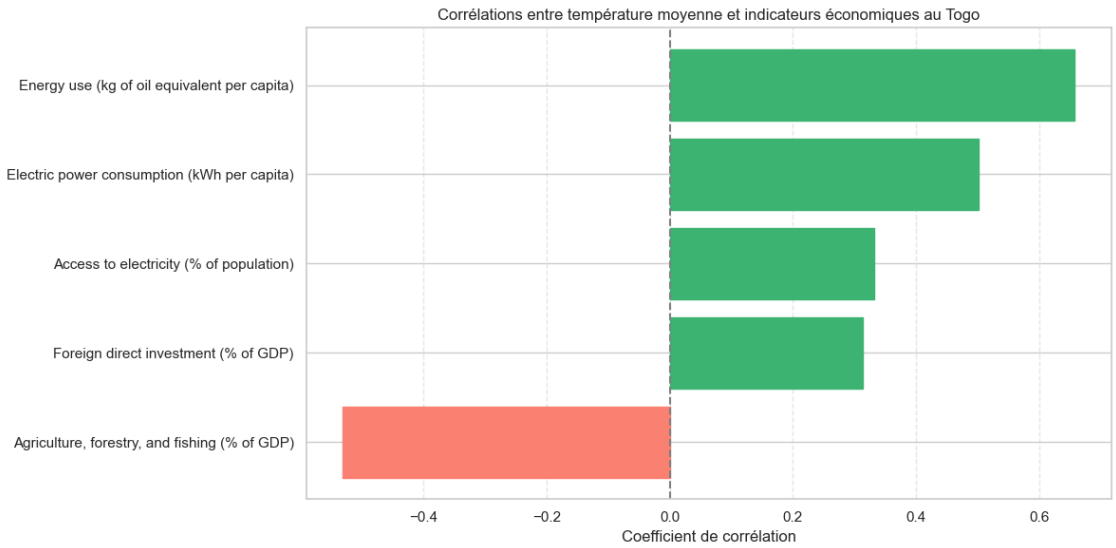
Meilleur accès à l'électricité,

Et plus d'investissements étrangers.

Cela suggère que le réchauffement climatique accompagne — ou accélère — une transformation économique, mais aussi une dépendance énergétique croissante, avec des risques pour les secteurs traditionnels comme l'agriculture.

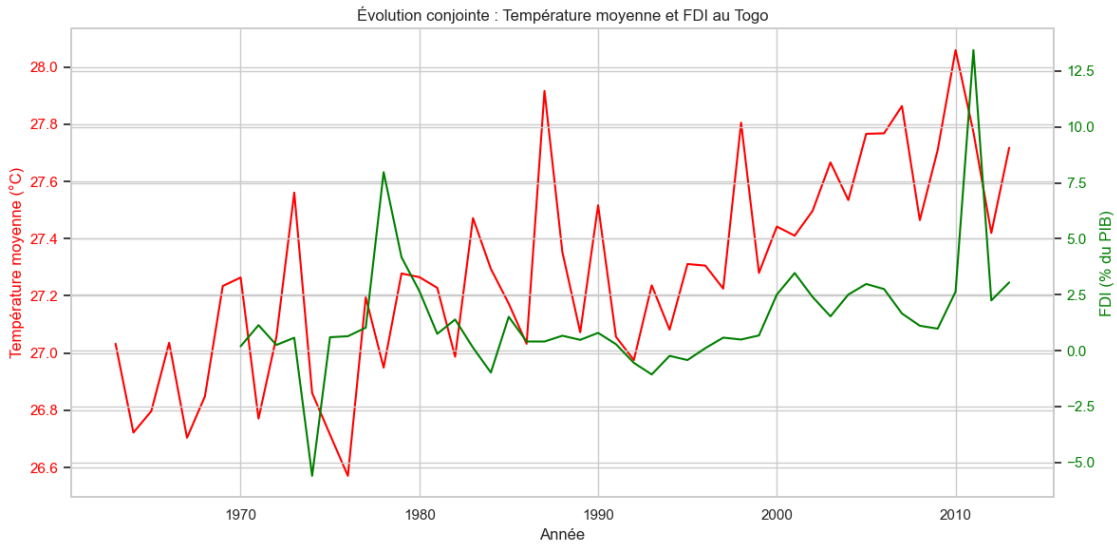
[298]:
```
# VISUALISATIONS DES CORRELATIONS

# Définition des corrélations
correlations = {
    "Foreign direct investment (% of GDP)": 0.313,
    "Agriculture, forestry, and fishing (% of GDP)": -0.531,
    "Energy use (kg of oil equivalent per capita)": 0.658,
    "Access to electricity (% of population)": 0.332,
    "Electric power consumption (kWh per capita)": 0.501
}

# Tri des corrélations par valeur croissante
correlations_sorted = dict(sorted(correlations.items(), key=lambda item:␣
 ↪item[1]))

# Création du barplot horizontal
plt.figure(figsize=(12, 6))
bars = plt.barh(list(correlations_sorted.keys()), list(correlations_sorted.
 ↪values()), color='skyblue')

# Mise en couleur selon le signe de la corrélation
for bar in bars:
    if bar.get_width() < 0:
        bar.set_color('salmon')  # Corrélations négatives
    else:
        bar.set_color('mediumseagreen')  # Corrélations positives

# Ligne verticale à zéro pour référence
plt.axvline(0, color='gray', linestyle='--')
```

```
# Mise en forme du graphique
plt.title("Corrélations entre température moyenne et indicateurs économiques au␣
↪Togo")
plt.xlabel("Coefficient de corrélation")
plt.grid(True, axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()

# Affichage
plt.show()
```



[299]:
```
# Visualisation conjointe de la température moyenne et de la FDI au Togo
fig, ax1 = plt.subplots(figsize=(12, 6))

# Axe 1 : Température
ax1.set_xlabel("Année")
ax1.set_ylabel("Température moyenne (°C)", color="red")
ax1.plot(df_merged["Year"], df_merged["AverageTemperature"], color="red",␣
↪label="Température")
ax1.tick_params(axis='y', labelcolor="red")

# Axe 2 : Indicateur économique
ax2 = ax1.twinx()
ax2.set_ylabel("FDI (% du PIB)", color="green")
ax2.plot(df_merged["Year"], df_merged["Foreign direct investment, net inflows␣
↪(% of GDP)"], color="green", label="IDE")
ax2.tick_params(axis='y', labelcolor="green")

plt.title("Évolution conjointe : Température moyenne et FDI au Togo")
```

```
fig.tight_layout()
plt.show()
```



Observation

FDI très instable

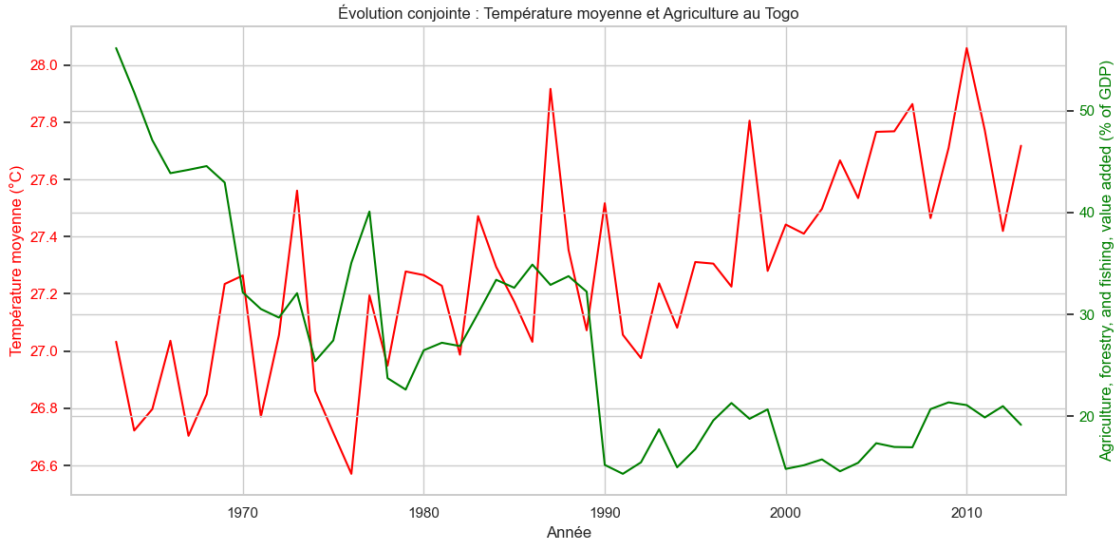
Température ↑ stable

Explication historique

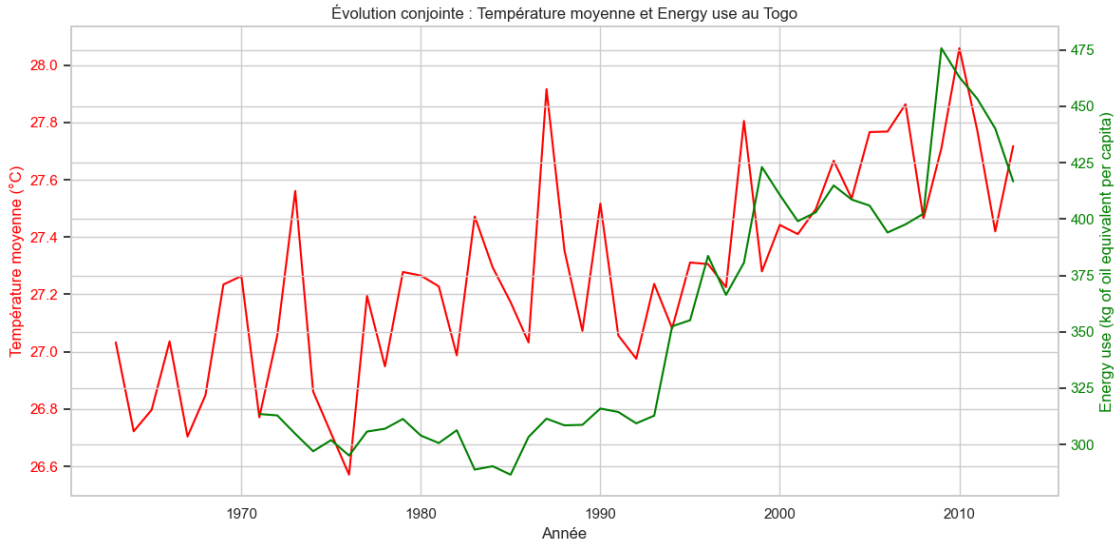
Période Événement 1990–1993 crise démocratique → chute FDI 2005 crise succession Eyadéma → chute FDI 2010 réformes → FDI remonte

[300]:
```
"visualisations conjointes de la température moyenne et agriculture , forestry,␣
↪and fishing, value added (% of GDP) au Togo"
fig, ax1 = plt.subplots(figsize=(12, 6))

# Axe 1 : Température
ax1.set_xlabel("Année")
ax1.set_ylabel("Température moyenne (°C)", color="red")
ax1.plot(df_merged["Year"], df_merged["AverageTemperature"], color="red",␣
↪label="Température")
ax1.tick_params(axis='y', labelcolor="red")

# Axe 2 : Indicateur économique
ax2 = ax1.twinx()
ax2.set_ylabel("Agriculture, forestry, and fishing, value added (% of GDP)",␣
↪color="green")
ax2.plot(df_merged["Year"], df_merged["Agriculture, forestry, and fishing,␣
↪value added (% of GDP)"], color="green", label="Agriculture")
```

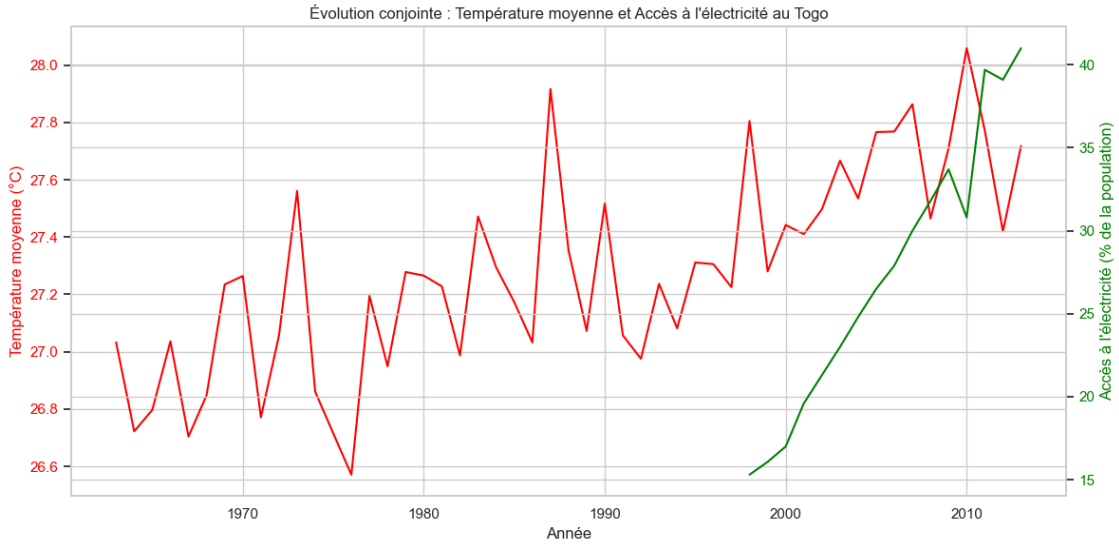
```
ax2.tick_params(axis='y', labelcolor="green")

plt.title("Évolution conjointe : Température moyenne et Agriculture au Togo")
fig.tight_layout()
plt.show()
```



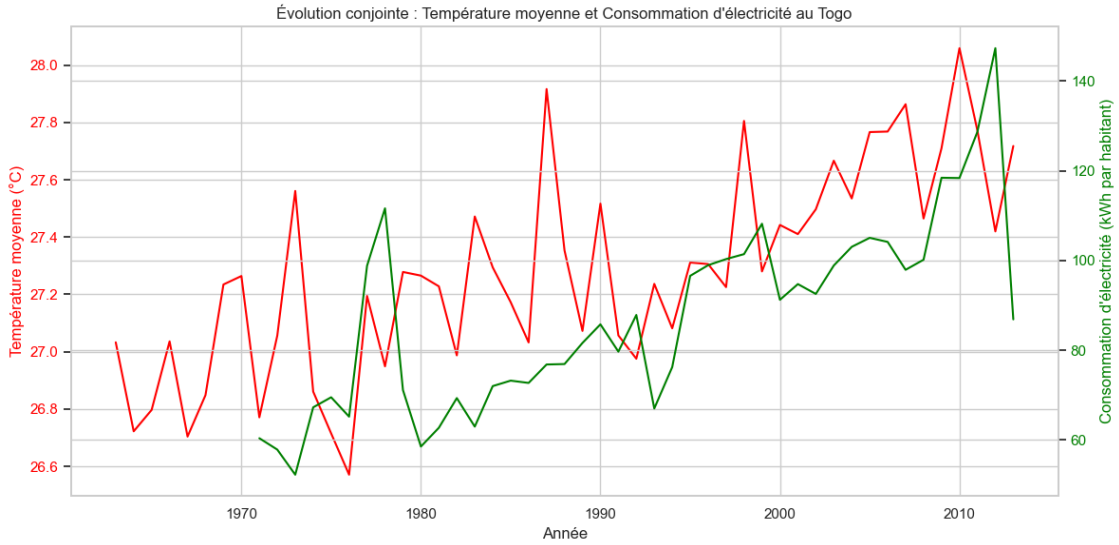
[301]:
```
#visualisations conjointes de la température moyenne et Energy use (kg of oil␣
↪equivalent per capita) au Togo
fig, ax1 = plt.subplots(figsize=(12, 6))
# Axe 1 : Température
ax1.set_xlabel("Année")
ax1.set_ylabel("Température moyenne (°C)", color="red")
ax1.plot(df_merged["Year"], df_merged["AverageTemperature"], color="red",␣
↪label="Température")
ax1.tick_params(axis='y', labelcolor="red")

# Axe 2 : Indicateur économique
ax2 = ax1.twinx()
ax2.set_ylabel("Energy use (kg of oil equivalent per capita)", color="green")
ax2.plot(df_merged["Year"], df_merged["Energy use (kg of oil equivalent per␣
↪capita)"], color="green", label="Energy use")
ax2.tick_params(axis='y', labelcolor="green")

plt.title("Évolution conjointe : Température moyenne et Energy use au Togo")
fig.tight_layout()
plt.show()
```



[302]:
```
#visualisations conjointes de la température moyenne et Access to electricity␣
↪(% of population) au Togo
fig, ax1 = plt.subplots(figsize=(12, 6))
# Axe 1 : Température
ax1.set_xlabel("Année")
ax1.set_ylabel("Température moyenne (°C)", color="red")
ax1.plot(df_merged["Year"], df_merged["AverageTemperature"], color="red",␣
↪label="Température")
ax1.tick_params(axis='y', labelcolor="red")

# Axe 2 : Indicateur économique
ax2 = ax1.twinx()
ax2.set_ylabel("Accès à l'électricité (% de la population)", color="green")
ax2.plot(df_merged["Year"], df_merged["Access to electricity (% of␣
↪population)"], color="green", label="Accès à l'électricité")
ax2.tick_params(axis='y', labelcolor="green")

plt.title("Évolution conjointe : Température moyenne et Accès à l'électricité␣
↪au Togo")
fig.tight_layout()
plt.show()
```



[303]:
```
#visualisations conjointes de la température moyenne et electricity consumption␣
↪(kWh per capita) au Togo
fig, ax1 = plt.subplots(figsize=(12, 6))
# Axe 1 : Température
ax1.set_xlabel("Année")
ax1.set_ylabel("Température moyenne (°C)", color="red")
ax1.plot(df_merged["Year"], df_merged["AverageTemperature"], color="red",␣
↪label="Température")
ax1.tick_params(axis='y', labelcolor="red")

# Axe 2 : Indicateur économique
ax2 = ax1.twinx()
ax2.set_ylabel("Consommation d'électricité (kWh par habitant)", color="green")
ax2.plot(df_merged["Year"], df_merged["Electric power consumption (kWh per␣
↪capita)"], color="green", label="Consommation d'électricité")
ax2.tick_params(axis='y', labelcolor="green")

plt.title("Évolution conjointe : Température moyenne et Consommation␣
↪d'électricité au Togo")
fig.tight_layout()
plt.show()
```



8 A. AXE 2 _ RESILIENCE AGRICOLE

8.0.1 Hypothèse : Malgré la hausse des émissions de CO , l'agriculture togolaise montre des signes de résilience à travers l'augmentation des rendements céréaliers et l'expansion des terres cultivées, même si son poids économique diminue.

[304]:
```
# Filtrer uniquement le Togo
df_co2_togo = df_c02[df_c02["Country Name"] == "Togo"]

# Transformation en format long
df_co2_long = df_co2_togo.melt(
    id_vars=["Country Name", "Country Code", "Indicator Name", "Indicator␣
 ↪Code"],
    var_name="Year",
    value_name="Value"
)

# Conversion de l'année en entier
df_co2_long["Year"] = df_co2_long["Year"].astype(int)

# Filtrer l'indicateur CO  par habitant
nom_indicateur_co2 = "Carbon dioxide (CO2) emissions excluding LULUCF per␣
 ↪capita (t CO2e/capita)"  # adapte si nécessaire
df_co2_filtered = df_co2_long[df_co2_long["Indicator Name"] ==␣
 ↪nom_indicateur_co2]

# Renommer la colonne pour plus de clarté
df_co2_filtered = df_co2_filtered.rename(columns={"Value": "CO2_per_capita"})
```

```
# Garder uniquement les colonnes utiles
df_co2_filtered = df_co2_filtered[["Year", "CO2_per_capita"]].dropna()
df_co2_filtered.head()
```

[304]:

	Year	CO2_per_capita
10	1970	0.155071
11	1971	0.160614
12	1972	0.175262
13	1973	0.149872
14	1974	0.135284

[305]:

```
# Filtrer les données agricoles
indicateurs_agricoles = [
    "Cereal yield (kg per hectare)",
    "Agriculture, forestry, and fishing, value added (% of GDP)",
    "Agricultural land (% of land area)"
]

df_agri = df_togo[df_togo["Indicator Name"].isin(indicateurs_agricoles)].copy()
df_agri["Year"] = pd.to_numeric(df_agri["Year"], errors="coerce").
 ↪astype("Int64")
df_agri["Value"] = pd.to_numeric(df_agri["Value"], errors="coerce")

# Pivot : année en index, indicateurs en colonnes
df_agri_pivot = df_agri.pivot_table(index="Year", columns="Indicator Name",␣
 ↪values="Value").reset_index()
```

[306]:

```
# Filtrer uniquement le Togo dans le dataset CO
df_co2_togo = df_c02[df_c02["Country Name"] == "Togo"].copy()

# Identifier les colonnes année (ex: "1960", "1961", ...)
colonnes_annees = [col for col in df_co2_togo.columns if col.isdigit()]

# Transformation en format long
df_co2_long = df_co2_togo.melt(
    id_vars=["Country Name", "Country Code", "Indicator Name", "Indicator␣
 ↪Code"],
    value_vars=colonnes_annees,
    var_name="Year",
    value_name="Value"
)

# Nettoyage
df_co2_long["Year"] = pd.to_numeric(df_co2_long["Year"], errors="coerce").
 ↪astype("Int64")
df_co2_long["Value"] = pd.to_numeric(df_co2_long["Value"], errors="coerce")
```

```
# Filtrer l'indicateur CO par habitant
nom_indicateur_co2 = "Carbon dioxide (CO2) emissions excluding LULUCF per␣
↪capita (t CO2e/capita)"
df_co2_filtered = df_co2_long[df_co2_long["Indicator Name"] ==␣
↪nom_indicateur_co2].copy()

# Renommer pour clarté
df_co2_filtered = df_co2_filtered.rename(columns={"Value": "CO2_per_capita"})
df_co2_filtered = df_co2_filtered[["Year", "CO2_per_capita"]].dropna()
```

[307]:
```
# Fusion propre sur la colonne "Year"
df_merged = pd.merge(
    df_agri_pivot,
    df_co2_filtered,
    on="Year",
    how="inner"
).dropna()
```

[308]:
```
# Corrélation entre CO et chaque indicateur agricole
correlations = {}
for col in indicateurs_agricoles:
    corr = df_merged["CO2_per_capita"].corr(df_merged[col])
    correlations[col] = round(corr, 3)
    print(f"Corrélation entre CO et {col} : {corr:.3f}")
```

```
Corrélation entre CO et Cereal yield (kg per hectare) : 0.702
Corrélation entre CO et Agriculture, forestry, and fishing, value added (% of
GDP) : -0.419
Corrélation entre CO et Agricultural land (% of land area) : 0.723
```

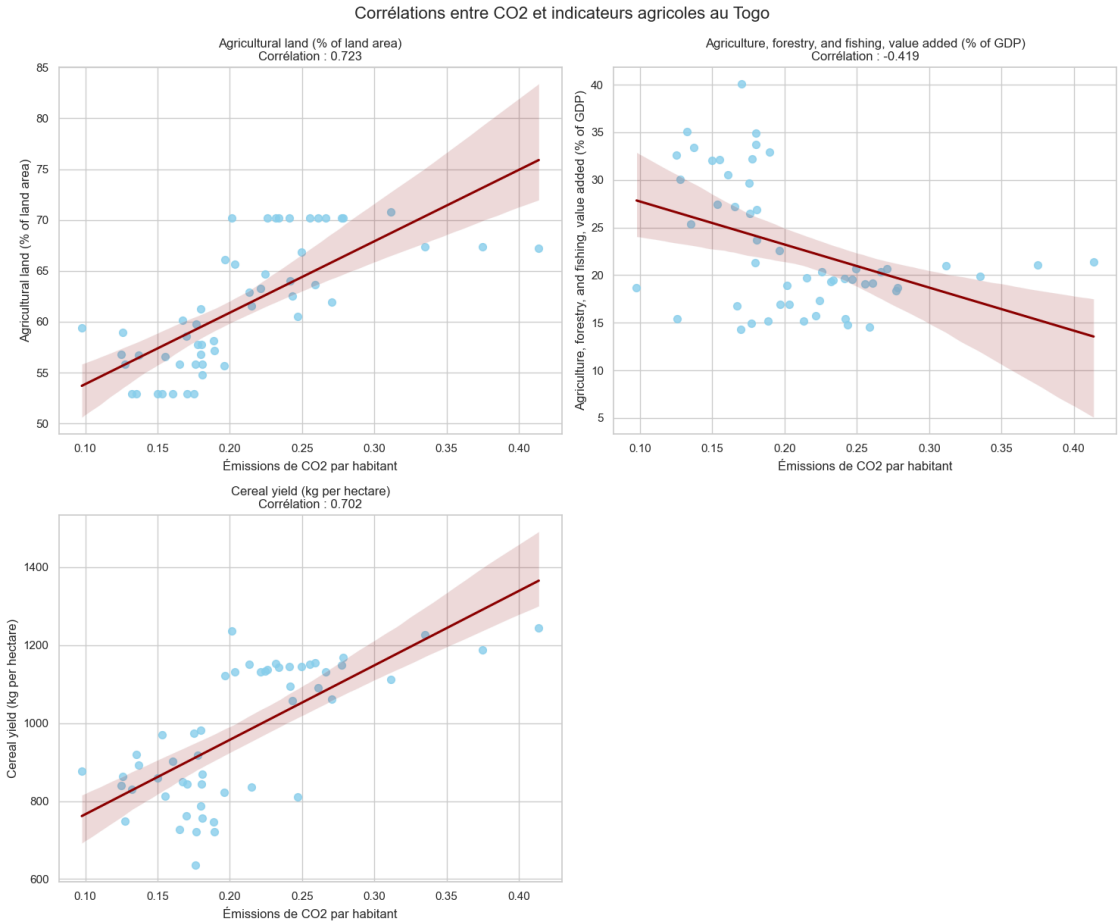
Cereal yield (kg per hectare) +0.702 Corrélation forte : les rendements céréaliers augmentent avec les émissions de CO , ce qui peut refléter une intensification agricole ou une adaptation technique (engrais, irrigation, variétés améliorées).

Agricultural land (% of land area) +0.723 Corrélation forte : les terres agricoles s'étendent avec les émissions, ce qui peut indiquer une réponse adaptative face aux pressions climatiques ou une volonté de renforcer la sécurité alimentaire.

Agriculture, forestry and fishing (% of GDP) –0.419 Corrélation négative : la part de l'agriculture dans le PIB diminue malgré l'expansion physique et les gains de productivité, ce qui peut refléter une transformation économique ou une perte de compétitivité du secteur.

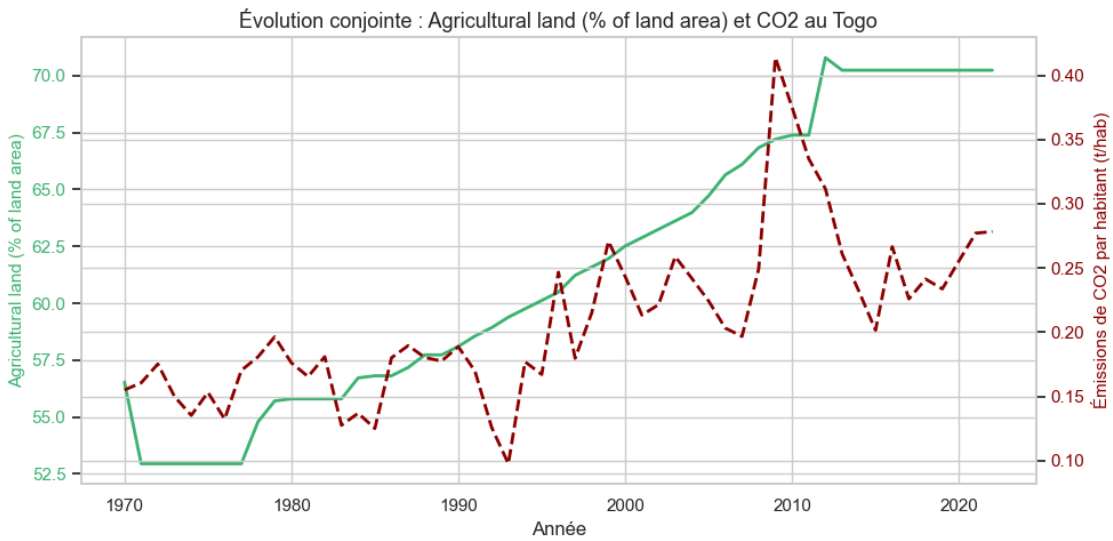
Les données confirment que l'agriculture togolaise s'adapte partiellement à la hausse des émissions de CO . Les rendements céréaliers augmentent et les terres cultivées s'étendent, ce qui montre une certaine capacité de réponse face aux pressions environnementales. Cependant, la baisse de la part de l'agriculture dans le PIB révèle une fragilité économique du secteur. Ainsi, bien que l'agriculture montre des signes de résilience sur le terrain, elle reste vulnérable dans sa contribution à l'économie nationale

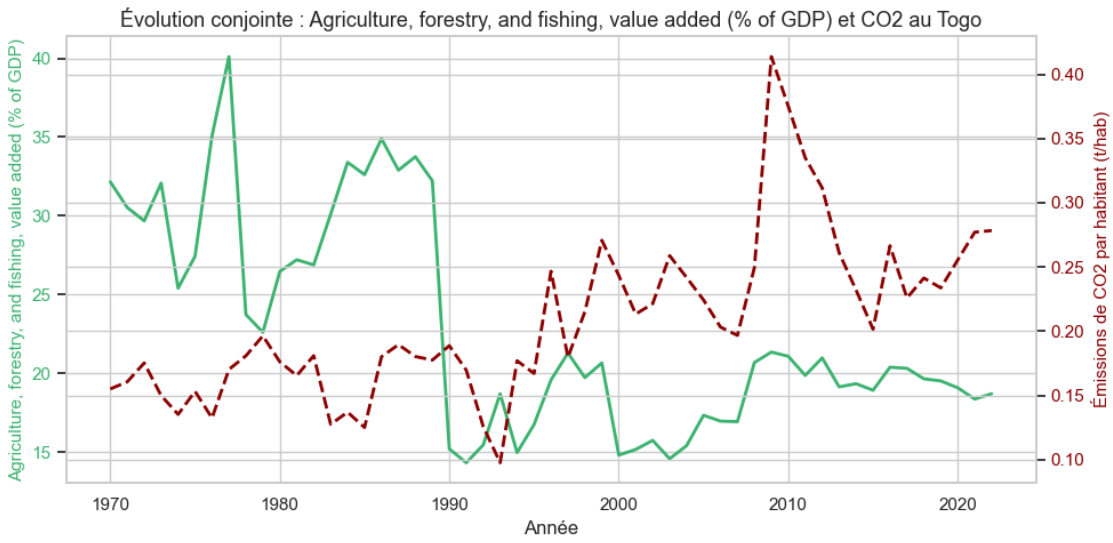
[309]:
```python
import matplotlib.pyplot as plt
import seaborn as sns

# Liste des indicateurs agricoles
indicateurs_agricoles = [
    "Agricultural land (% of land area)",
    "Agriculture, forestry, and fishing, value added (% of GDP)",
    "Cereal yield (kg per hectare)"
]

# Style visuel
sns.set(style="whitegrid")
plt.figure(figsize=(15, 12))

# Boucle sur chaque indicateur
for i, indicateur in enumerate(indicateurs_agricoles, 1):
    plt.subplot(2, 2, i)

    # Scatter plot + ligne de régression
    sns.regplot(
        x="CO2_per_capita",
        y=indicateur,
        data=df_merged,
        scatter_kws={"s": 50, "color": "skyblue"},
        line_kws={"color": "darkred"}
    )

    # Calcul de la corrélation
    corr = df_merged["CO2_per_capita"].corr(df_merged[indicateur])

    # Annotation
    plt.title(f"{indicateur}\nCorrélation : {round(corr, 3)}", fontsize=12)
    plt.xlabel("Émissions de CO2 par habitant")
    plt.ylabel(indicateur)

plt.tight_layout()
plt.suptitle("Corrélations entre CO2 et indicateurs agricoles au Togo",␣
 ↪fontsize=16, y=1.02)
plt.show()
```

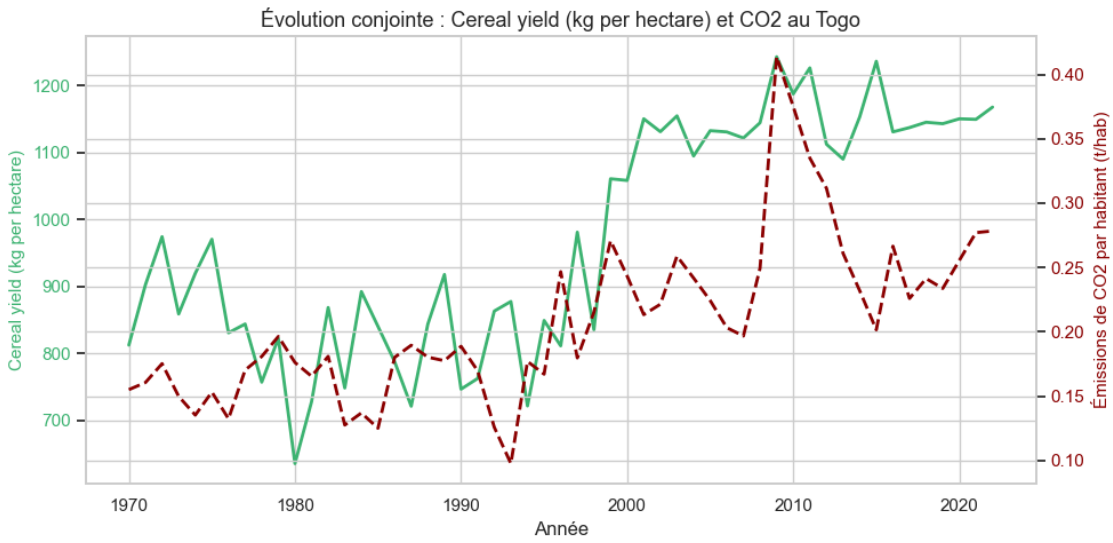


[311]:
```
import seaborn as sns
sns.set_style("whitegrid")

# Liste des indicateurs agricoles
indicateurs_agricoles = [
    "Agricultural land (% of land area)",
    "Agriculture, forestry, and fishing, value added (% of GDP)",
    "Cereal yield (kg per hectare)"
]


# Création d'un graphique par indicateur
for indicateur in indicateurs_agricoles:
    fig, ax1 = plt.subplots(figsize=(10, 5))

    # Axe Y gauche : indicateur agricole
    ax1.plot(df_merged["Year"], df_merged[indicateur], color='mediumseagreen',␣
 ↪linewidth=2, label=indicateur)
```

```
    ax1.set_ylabel(indicateur, color='mediumseagreen', fontsize=11)
    ax1.tick_params(axis='y', labelcolor='mediumseagreen')

    # Axe Y droit : CO2
    ax2 = ax1.twinx()
    ax2.plot(df_merged["Year"], df_merged["CO2_per_capita"], color='darkred',␣
↪linestyle='--', linewidth=2, label='CO2 per capita')
    ax2.set_ylabel("Émissions de CO2 par habitant (t/hab)", color='darkred',␣
↪fontsize=11)
    ax2.tick_params(axis='y', labelcolor='darkred')

    # Titre et mise en forme
    plt.title(f"Évolution conjointe : {indicateur} et CO2 au Togo", fontsize=13)
    ax1.set_xlabel("Année")
    fig.tight_layout()
    plt.show()
```







8.0.2 Hypothèse :Quand les températures augmentent, l'agriculture au Togo s'adapte : les récoltes de céréales deviennent plus abondantes et les terres cultivées s'élargissent. Mais en même temps, l'agriculture représente une part plus petite dans l'économie du pays.

[ ]:
```
#préparation des données de température pour le Togo

# Convertir la date en datetime
df_temp_cleaned["dt"] = pd.to_datetime(df_temp_cleaned["dt"])
```

```
# Extraire l'année
df_temp_cleaned["Year"] = df_temp_cleaned["dt"].dt.year

# Filtrer le Togo uniquement
df_temp_togo = df_temp_cleaned[df_temp_cleaned["Country"] == "Togo"]

# Agréger par année : moyenne des températures par ville
df_temp_yearly = df_temp_togo.groupby("Year")["AverageTemperature"].mean().
 ↪reset_index()
```

```
C:\Users\fangn\AppData\Local\Temp\ipykernel_15492\3321265343.py:4:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df_temp_cleaned["dt"] = pd.to_datetime(df_temp_cleaned["dt"])
C:\Users\fangn\AppData\Local\Temp\ipykernel_15492\3321265343.py:7:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df_temp_cleaned["Year"] = df_temp_cleaned["dt"].dt.year
```

[ ]:
```
# Liste des indicateurs agricoles pertinents
indicateurs_agricoles = [
    "Cereal yield (kg per hectare)",
    "Agriculture, forestry, and fishing, value added (% of GDP)",
    "Agricultural land (% of land area)"
]

# Filtrer les données agricoles
df_agri = df_togo[df_togo["Indicator Name"].isin(indicateurs_agricoles)]

# Pivot : année en index, indicateurs en colonnes
df_agri_pivot = df_agri.pivot_table(index="Year", columns="Indicator Name",␣
 ↪values="Value")
df_agri_pivot = df_agri_pivot.reset_index()
```

[ ]:
```
df_agri_pivot.index = df_agri_pivot.index.astype(int)
```

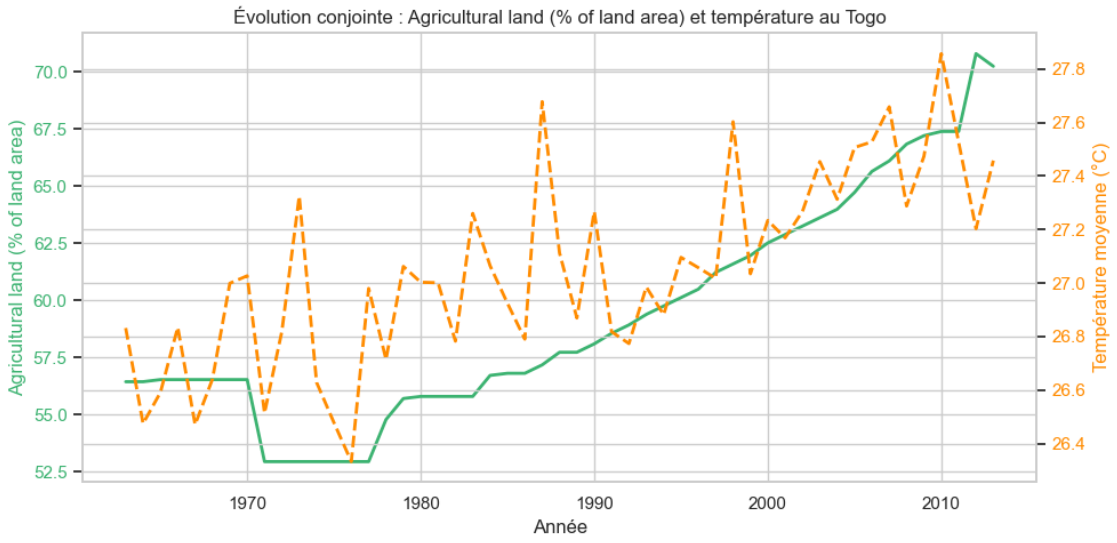
[ ]:
```
df_agri_pivot["Year"] = pd.to_numeric(df_agri_pivot["Year"], errors="coerce").
 ↪astype("Int64")
```

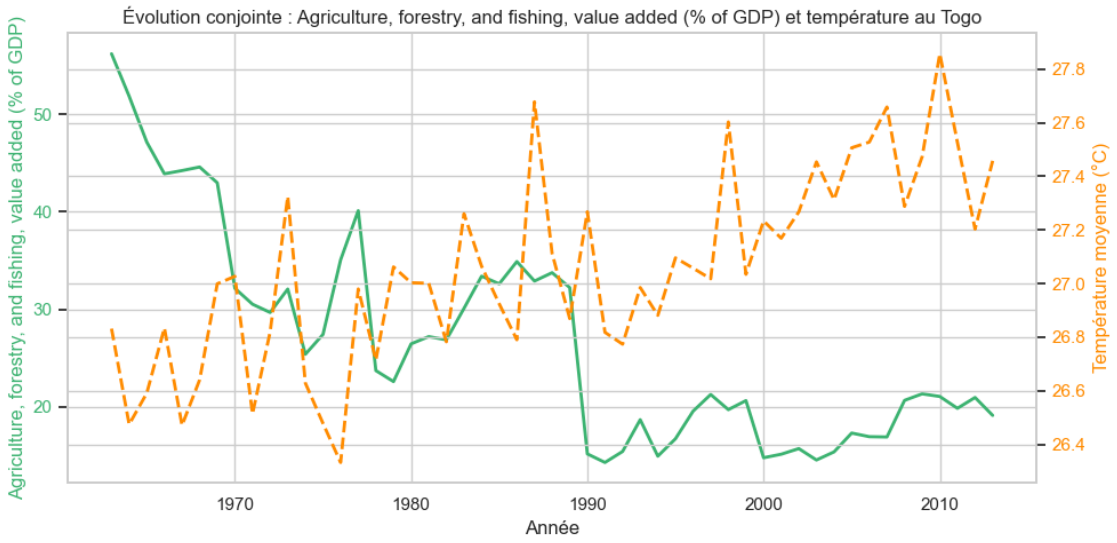
```
df_temp_yearly.index = pd.to_numeric(df_temp_yearly.index, errors="coerce").
 ↪astype("Int64")
```

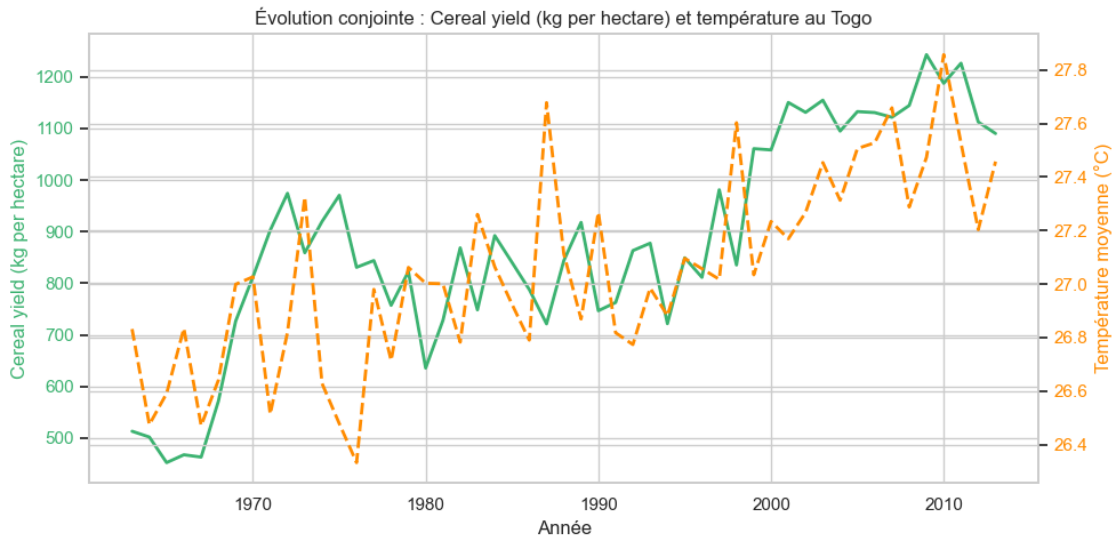
[ ]:
```
# Si l'année est déjà dans l'index de df_temp_yearly
df_temp_yearly = df_temp_yearly.reset_index()

# Assure-toi que les années sont bien au format entier
df_temp_yearly["Year"] = pd.to_numeric(df_temp_yearly["Year"], errors="coerce").
 ↪astype("Int64")
df_agri_pivot["Year"] = pd.to_numeric(df_agri_pivot["Year"], errors="coerce").
 ↪astype("Int64")

# Fusion propre sur la colonne Year
df_temp_agri = pd.merge(
    df_agri_pivot,
    df_temp_yearly,
    on="Year",
    how="inner"
).dropna()
```

[ ]:
```
# Visualisation conjointe de la température moyenne et des indicateurs␣
 ↪agricoles au Togo


# Exclure "Year" s'il est présent dans les colonnes
indicateurs_agricoles = [col for col in df_agri_pivot.columns if col.lower() !=␣
 ↪"year"]

for indicateur in indicateurs_agricoles:
    fig, ax1 = plt.subplots(figsize=(10, 5))

    # Axe Y gauche : indicateur agricole
    ax1.plot(df_temp_agri["Year"], df_temp_agri[indicateur],␣
 ↪color='mediumseagreen', linewidth=2)
    ax1.set_ylabel(indicateur, color='mediumseagreen')
    ax1.tick_params(axis='y', labelcolor='mediumseagreen')

    # Axe Y droit : température
    ax2 = ax1.twinx()
    ax2.plot(df_temp_agri["Year"], df_temp_agri["AverageTemperature"],␣
 ↪color='darkorange', linestyle='--', linewidth=2)
    ax2.set_ylabel("Température moyenne (°C)", color='darkorange')
    ax2.tick_params(axis='y', labelcolor='darkorange')

    plt.title(f"Évolution conjointe : {indicateur} et température au Togo")
    ax1.set_xlabel("Année")
    fig.tight_layout()
```

```
plt.show()
```


Évolution conjointe : Agricultural land (% of land area) et température au Togo
Agricultural land (% of land area)
Température moyenne (°C)
Année
70.0
67.5
65.0
62.5
60.0
57.5
55.0
52.5
27.8
27.6
27.4
27.2
27.0
26.8
26.6
26.4
1970
1980
1990
2000
2010


Évolution conjointe : Agriculture, forestry, and fishing, value added (% of GDP) et température au Togo
Agriculture, forestry, and fishing, value added (% of GDP)
Température moyenne (°C)
Année
50
40
30
20
27.8
27.6
27.4
27.2
27.0
26.8
26.6
26.4
1970
1980
1990
2000
2010



[ ]:
```python
# Liste des indicateurs agricoles
indicateurs_agricoles = df_agri_pivot.columns.tolist()

# Exclure "Year" si elle est présente
indicateurs_agricoles = [col for col in indicateurs_agricoles if col.lower() !=␣
↪"year"]

# Dictionnaire pour stocker les corrélations
correlations_temp = {}

# Calcul des corrélations avec la température
for indicateur in indicateurs_agricoles:
    corr = df_temp_agri["AverageTemperature"].corr(df_temp_agri[indicateur])
    correlations_temp[indicateur] = round(corr, 3)

# Affichage des résultats
print(" Corrélations entre température moyenne et indicateurs agricoles au Togo␣
↪:")
for indicateur, corr in correlations_temp.items():
    print(f"- {indicateur} : {corr}")
```

```
 Corrélations entre température moyenne et indicateurs agricoles au Togo :
- Agricultural land (% of land area) : 0.707
- Agriculture, forestry, and fishing, value added (% of GDP) : -0.527
- Cereal yield (kg per hectare) : 0.608
```

Agricultural land (% of land area) +0.707 Corrélation forte : les terres agricoles augmentent avec la température, ce qui peut indiquer une stratégie d'adaptation pour compenser les pertes ou répondre à une demande croissante.

Cereal yield (kg per hectare) +0.608 Corrélation modérée à forte : les rendements céréaliers progressent malgré la hausse des températures, ce qui suggère une amélioration des pratiques agricoles ou une adaptation technique.

Agriculture, forestry and fishing (% of GDP) –0.527 Corrélation négative : la part de l'agriculture dans le PIB diminue, ce qui peut refléter une transformation économique ou une perte de compétitivité du secteur malgré les efforts d'adaptation.

Les données confirment que l'agriculture togolaise développe une forme de résilience face à la hausse des températures. En effet, les rendements céréaliers augmentent et les terres cultivées s'étendent, ce qui montre une capacité d'adaptation du secteur. Toutefois, la baisse de la part de l'agriculture dans le PIB indique que cette résilience reste principalement technique et ne se traduit pas encore par un renforcement économique du secteur. Cela souligne la nécessité d'accompagner les efforts agricoles par des politiques de valorisation et de soutien durable.

9 A. AXE 2 _ la condition de vie

9.0.1 Hypothèse : La hausse des températures au Togo accompagne une amélioration partielle des conditions de vie en milieu urbain, mais elle s'accompagne aussi d'une forte vulnérabilité sociale et environnementale pour les populations les plus exposées.

[ ]:
```
# Préparation d'un tableau combiné des indicateurs de vie et de la température␣
↪au Togo

# Étape 1 - Liste des indicateurs à extraire
indicateurs_vie = [
    'Access to electricity (% of population)',
    'Electric power consumption (kWh per capita)',
    'Renewable energy consumption (% of total final energy consumption)',
    'Mortality rate, under-5 (per 1,000 live births)',
    'Prevalence of underweight, weight for age (% of children under 5)',
    'Primary completion rate, total (% of relevant age group)',
    'School enrollment, primary and secondary (gross), gender parity index␣
↪(GPI)',
    'Poverty headcount ratio at $3.00 a day (2021 PPP) (% of population)',
    'Urban population (% of total population)',
    'Population in urban agglomerations of more than 1 million (% of total␣
↪population)',
    'Population living in areas where elevation is below 5 meters (% of total␣
↪population)',
    'Average precipitation in depth (mm per year)',
    'Droughts, floods, extreme temperatures (% of population, average␣
↪1990-2009)'
]

# Étape 2 - Filtrer les indicateurs dans df_togo
df_vie = df_togo[df_togo["Indicator Name"].isin(indicateurs_vie)]
```

```python
# Étape 3 - Pivot : années en index, indicateurs en colonnes
df_vie_pivot = df_vie.pivot_table(index="Year", columns="Indicator Name",␣
↪values="Value")

# Étape 4 - Préparer les températures annuelles
df_temp["dt"] = pd.to_datetime(df_temp["dt"])
df_temp["Year"] = df_temp["dt"].dt.year
df_temp_togo = df_temp[df_temp["Country"] == "Togo"]
df_temp_yearly = df_temp_togo.groupby("Year")["AverageTemperature"].mean().
↪to_frame()

# Étape 5 - Harmoniser les années
df_vie_pivot.index = df_vie_pivot.index.astype(int)
df_temp_yearly.index = df_temp_yearly.index.astype(int)
annees_communes = df_vie_pivot.index.intersection(df_temp_yearly.index)
df_vie_pivot = df_vie_pivot.loc[annees_communes]
df_temp_yearly = df_temp_yearly.loc[annees_communes]

# Étape 6 - Fusion des deux sources
df_vie_temp = df_vie_pivot.merge(df_temp_yearly, left_index=True,␣
↪right_index=True)

# Étape 7 - Nettoyage : garder les colonnes avec moins de 50% de NaN
df_vie_temp_clean = df_vie_temp.loc[:, df_vie_temp.isna().mean() < 0.5]

# Étape 8 - Supprimer les lignes incomplètes
df_vie_temp_clean = df_vie_temp_clean.dropna()

#  Résultat final
print(" Tableau final prêt :", df_vie_temp_clean.shape)
df_vie_temp_clean
```

```
Tableau final prêt : (27, 8)
```

[ ]:

```
      Average precipitation in depth (mm per year)  \
Year
1971                                          1168.0
1972                                          1168.0
1976                                          1168.0
1978                                          1168.0
1979                                          1168.0
1980                                          1168.0
1982                                          1168.0
1983                                          1168.0
1984                                          1168.0
1985                                          1168.0
```

1986	1168.0
1987	1168.0
1988	1168.0
1989	1168.0
1990	1168.0
1991	1168.0
1992	1168.0
1994	1168.0
1995	1168.0
1996	1168.0
1999	1168.0
2000	1168.0
2001	1168.0
2003	1168.0
2004	1168.0
2005	1168.0
2006	1168.0

Year	Electric power consumption (kWh per capita) \
1971	60.397519
1972	57.861781
1976	65.205952
1978	111.626979
1979	71.130417
1980	58.543720
1982	69.340959
1983	62.986771
1984	72.010722
1985	73.232588
1986	72.718007
1987	76.828267
1988	76.928342
1989	81.629269
1990	85.776569
1991	79.697538
1992	87.868128
1994	76.250195
1995	96.580418
1996	98.954533
1999	108.194179
2000	91.244479
2001	94.743947
2003	98.880311
2004	103.045585
2005	105.053681
2006	104.148158

Year	Mortality rate, under-5 (per 1,000 live births) \
1971	220.5
1972	215.9
1976	197.0
1978	187.7
1979	183.2
1980	178.8
1982	170.6
1983	166.8
1984	163.2
1985	159.8
1986	156.7
1987	154.0
1988	151.5
1989	149.3
1990	147.2
1991	145.2
1992	143.3
1994	138.8
1995	136.2
1996	133.2
1999	123.2
2000	119.6
2001	116.0
2003	109.1
2004	105.9
2005	102.7
2006	99.7

Year	Population in urban agglomerations of more than 1 million (% of total population) \
1971	8.765923
1972	9.033398
1976	10.320093
1978	11.053383
1979	11.423158
1980	11.775984
1982	12.429688
1983	12.775641
1984	13.138806
1985	13.531377
1986	13.956108
1987	14.396660
1988	14.855128

1989	15.161929
1990	15.482896
1991	15.818260
1992	16.170216
1994	17.922656
1995	18.042900
1996	18.281184
1999	19.456873
2000	19.904001
2001	20.382985
2003	21.421186
2004	21.955187
2005	22.499282
2006	22.359844

Primary completion rate, total (% of relevant age group) \

Year	
1971	23.700991
1972	28.772610
1976	40.374962
1978	49.098080
1979	48.836769
1980	50.588100
1982	45.091129
1983	44.259399
1984	38.423191
1985	32.572281
1986	32.461731
1987	31.229691
1988	36.295448
1989	29.505569
1990	35.518959
1991	35.654869
1992	36.437141
1994	30.113529
1995	42.019089
1996	48.285919
1999	57.408520
2000	66.567574
2001	73.113472
2003	72.183403
2004	73.616257
2005	72.643028
2006	72.090691

School enrollment, primary and secondary (gross), gender parity index (GPI) \

Year	
1971	0.43126
1972	0.44686
1976	0.49106
1978	0.51916
1979	0.53722
1980	0.54691
1982	0.56918
1983	0.57471
1984	0.56863
1985	0.56121
1986	0.55208
1987	0.55450
1988	0.55949
1989	0.56563
1990	0.57858
1991	0.58714
1992	0.59324
1994	0.59504
1995	0.60884
1996	0.61685
1999	0.66992
2000	0.69135
2001	0.70089
2003	0.72190
2004	0.73548
2005	0.74424
2006	0.75646

Year	Urban population (% of total population)	AverageTemperature
1971	21.602	26.513750
1972	21.929	26.819583
1976	23.268	26.332625
1978	23.958	26.714375
1979	24.308	27.062042
1980	24.663	27.002792
1982	25.391	26.782667
1983	25.778	27.260250
1984	26.169	27.064333
1985	26.562	26.924208
1986	26.960	26.790292
1987	27.362	27.677833
1988	27.768	27.112208
1989	28.176	26.869208
1990	28.589	27.267625
1991	29.005	26.817917

```
1992                                    29.425          26.773750
1994                                    30.275          26.881333
1995                                    30.705          27.095792
1996                                    31.139          27.057250
1999                                    32.460          27.034292
2000                                    32.907          27.233625
2001                                    33.356          27.168875
2003                                    34.265          27.453833
2004                                    34.724          27.312750
2005                                    35.185          27.506333
2006                                    35.649          27.527542
```

[ ]:
```python
correlations_vie = {}
for indicateur in indicateurs_vie:
    if indicateur in df_vie_temp.columns:
        corr = df_vie_temp["AverageTemperature"].corr(df_vie_temp[indicateur])
        correlations_vie[indicateur] = round(corr, 3)

# Affichage
for k, v in correlations_vie.items():
    print(f"{k} : {v}")
```

```
Access to electricity (% of population) : 0.311
Electric power consumption (kWh per capita) : 0.51
Renewable energy consumption (% of total final energy consumption) : -0.576
Mortality rate, under-5 (per 1,000 live births) : -0.758
Prevalence of underweight, weight for age (% of children under 5) : -0.063
Primary completion rate, total (% of relevant age group) : 0.647
School enrollment, primary and secondary (gross), gender parity index (GPI) :
0.684
Poverty headcount ratio at $3.00 a day (2021 PPP) (% of population) : 1.0
Urban population (% of total population) : 0.747
Population in urban agglomerations of more than 1 million (% of total
population) : 0.761
Population living in areas where elevation is below 5 meters (% of total
population) : 1.0
Average precipitation in depth (mm per year) : nan
Droughts, floods, extreme temperatures (% of population, average 1990-2009) :
nan
```

```
c:\Users\fangn\OneDrive\Bureau\BACHELOR 2 NEXA DIGITAL SCHOOL\SEMESTRE
1\EVOLUTION DU CLIMAT  CERVELLO\env\Lib\site-
packages\numpy\lib\_function_base_impl.py:3065: RuntimeWarning: invalid value
encountered in divide
  c /= stddev[:, None]
c:\Users\fangn\OneDrive\Bureau\BACHELOR 2 NEXA DIGITAL SCHOOL\SEMESTRE
1\EVOLUTION DU CLIMAT  CERVELLO\env\Lib\site-
packages\numpy\lib\_function_base_impl.py:3066: RuntimeWarning: invalid value
```

```
encountered in divide
  c /= stddev[None, :]
c:\Users\fangn\OneDrive\Bureau\BACHELOR 2 NEXA DIGITAL SCHOOL\SEMESTRE
1\EVOLUTION DU CLIMAT  CERVELLO\env\Lib\site-
packages\numpy\lib\_function_base_impl.py:3057: RuntimeWarning: Degrees of
freedom <= 0 for slice
  c = cov(x, y, rowvar, dtype=dtype)
c:\Users\fangn\OneDrive\Bureau\BACHELOR 2 NEXA DIGITAL SCHOOL\SEMESTRE
1\EVOLUTION DU CLIMAT  CERVELLO\env\Lib\site-
packages\numpy\lib\_function_base_impl.py:2914: RuntimeWarning: divide by zero
encountered in divide
  c *= np.true_divide(1, fact)
c:\Users\fangn\OneDrive\Bureau\BACHELOR 2 NEXA DIGITAL SCHOOL\SEMESTRE
1\EVOLUTION DU CLIMAT  CERVELLO\env\Lib\site-
packages\numpy\lib\_function_base_impl.py:2914: RuntimeWarning: invalid value
encountered in multiply
  c *= np.true_divide(1, fact)
```

Primary completion rate (% d'enfants qui terminent le primaire) : +0.647 → L'éducation progresse avec la température.

Urban population (% de la population totale) : +0.747 → L'urbanisation augmente avec la température.

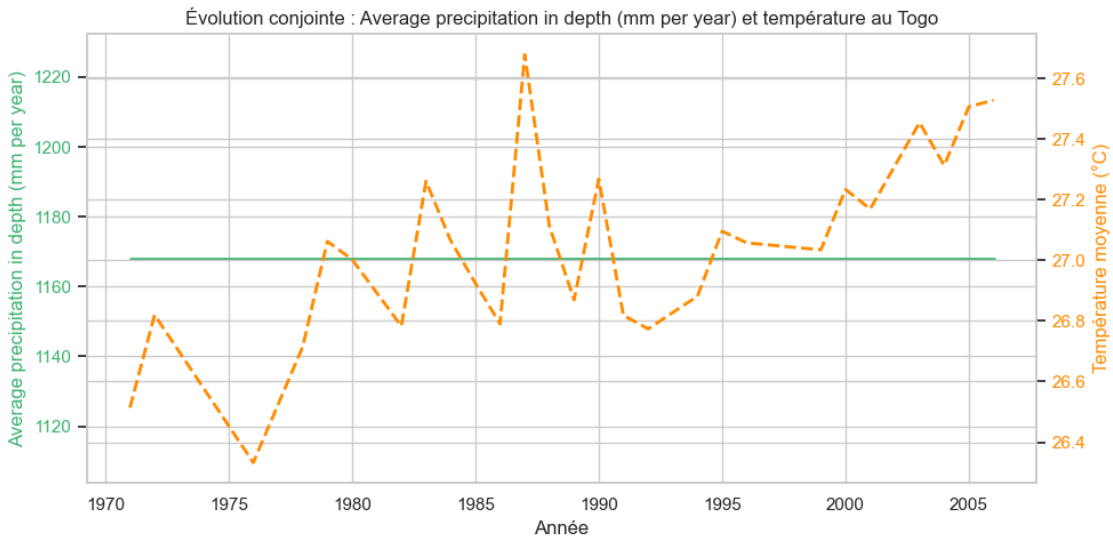
Mortality rate, under-5 (taux de mortalité des enfants de moins de 5 ans) : –0.758 → La mortalité infantile diminue, signe d'une amélioration sanitaire.

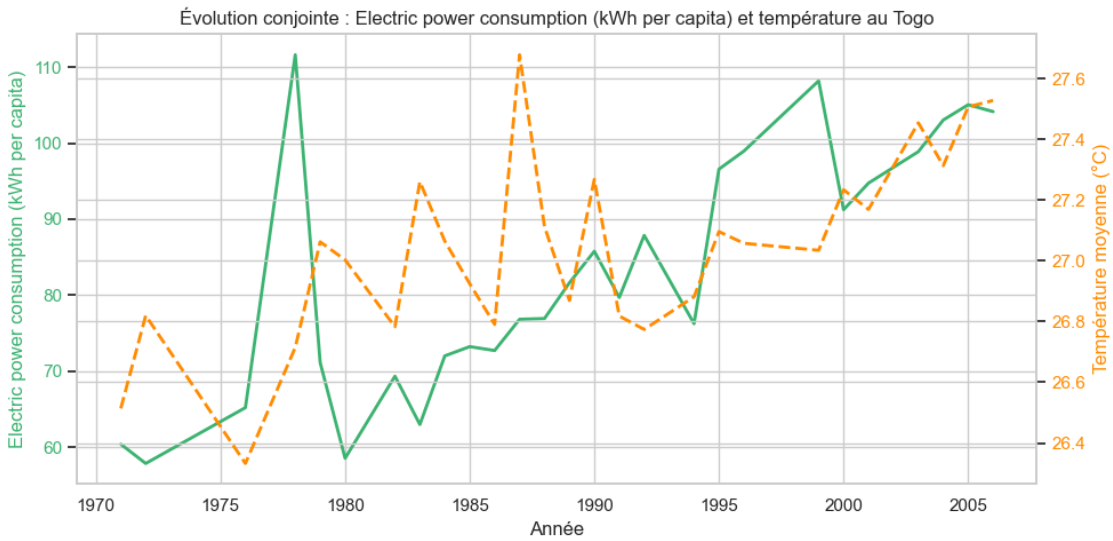
Poverty headcount ratio à $3/jour : +1.000 → La pauvreté reste très élevée malgré les autres progrès.

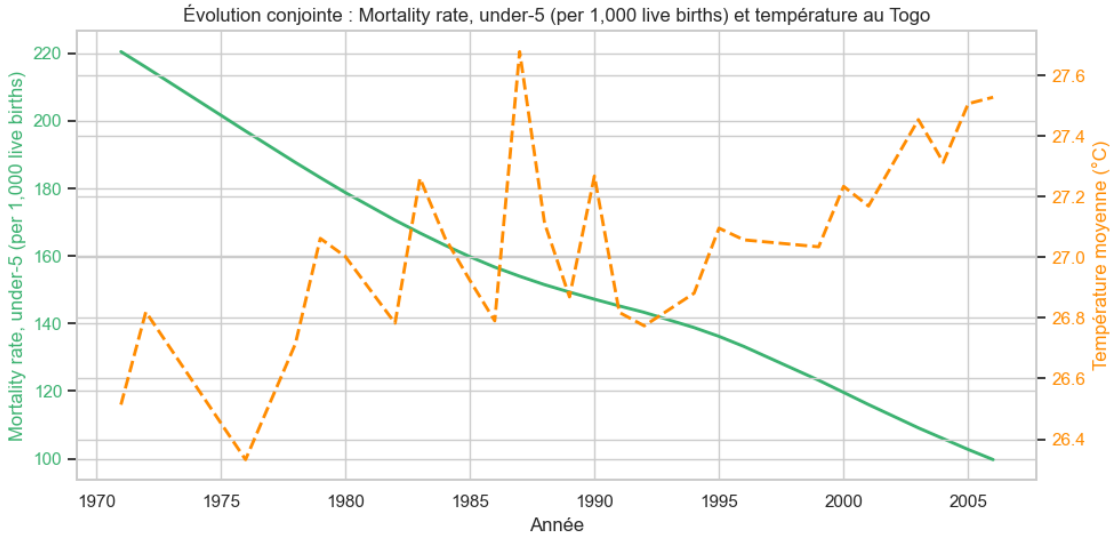
Les corrélations montrent que la hausse des températures au Togo est liée à certains progrès dans les conditions de vie, notamment en milieu urbain : meilleure éducation, accès à l'électricité, baisse de la mortalité infantile. Mais en parallèle, la pauvreté reste très élevée, les populations sont exposées à des risques climatiques, et la transition énergétique est insuffisante. Cela révèle une résilience partielle, mais aussi des inégalités et des vulnérabilités importantes.

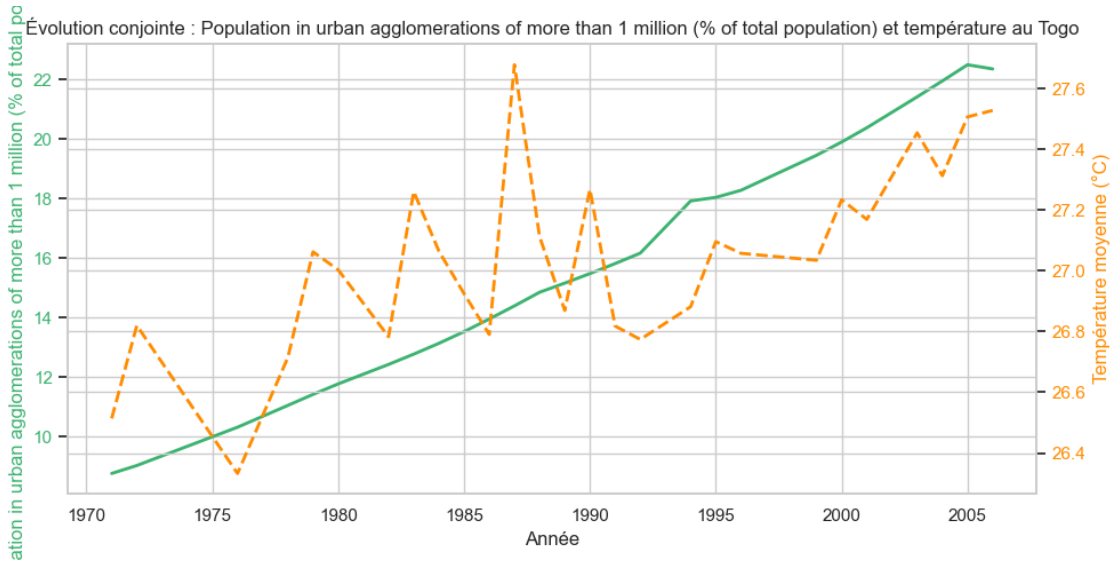
[ ]:
```python
# Liste des indicateurs à tracer (exclure 'AverageTemperature')
indicateurs_vie_temp = [col for col in df_vie_temp_clean.columns if col !=␣
↪"AverageTemperature"]

# Boucle sur chaque indicateur
for indicateur in indicateurs_vie_temp:
    fig, ax1 = plt.subplots(figsize=(10, 5))

    # Axe Y gauche : indicateur de vie
    ax1.plot(df_vie_temp_clean.index, df_vie_temp_clean[indicateur],␣
↪color='mediumseagreen', linewidth=2)
    ax1.set_ylabel(indicateur, color='mediumseagreen')
    ax1.tick_params(axis='y', labelcolor='mediumseagreen')
```

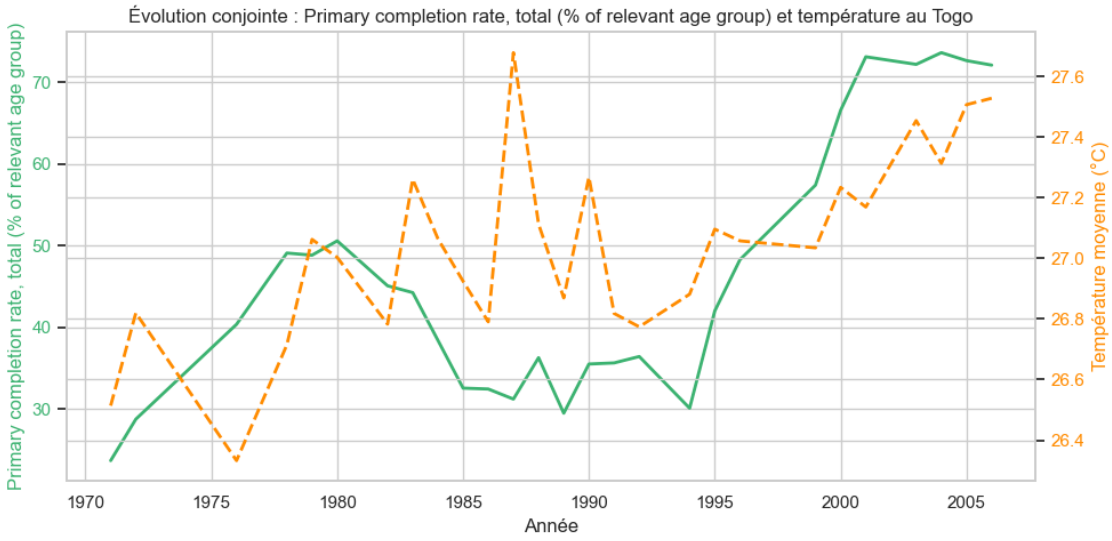
```
    # Axe Y droit : température
    ax2 = ax1.twinx()
    ax2.plot(df_vie_temp_clean.index, df_vie_temp_clean["AverageTemperature"],␣
↪color='darkorange', linestyle='--', linewidth=2)
    ax2.set_ylabel("Température moyenne (°C)", color='darkorange')
    ax2.tick_params(axis='y', labelcolor='darkorange')

    # Titre et mise en page
    plt.title(f"Évolution conjointe : {indicateur} et température au Togo")
    ax1.set_xlabel("Année")
    fig.tight_layout()
    plt.show()
```

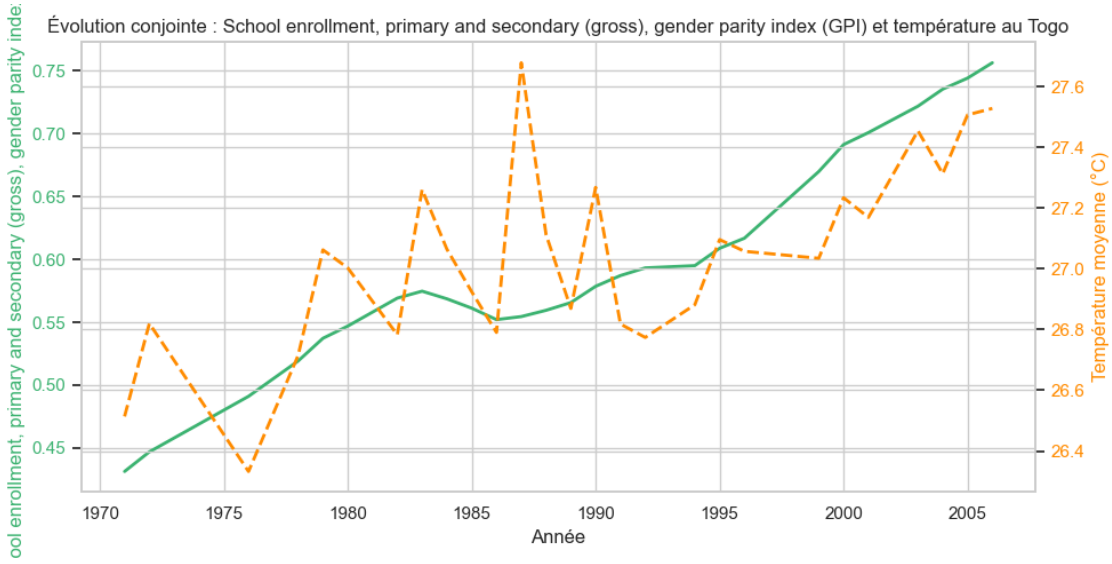


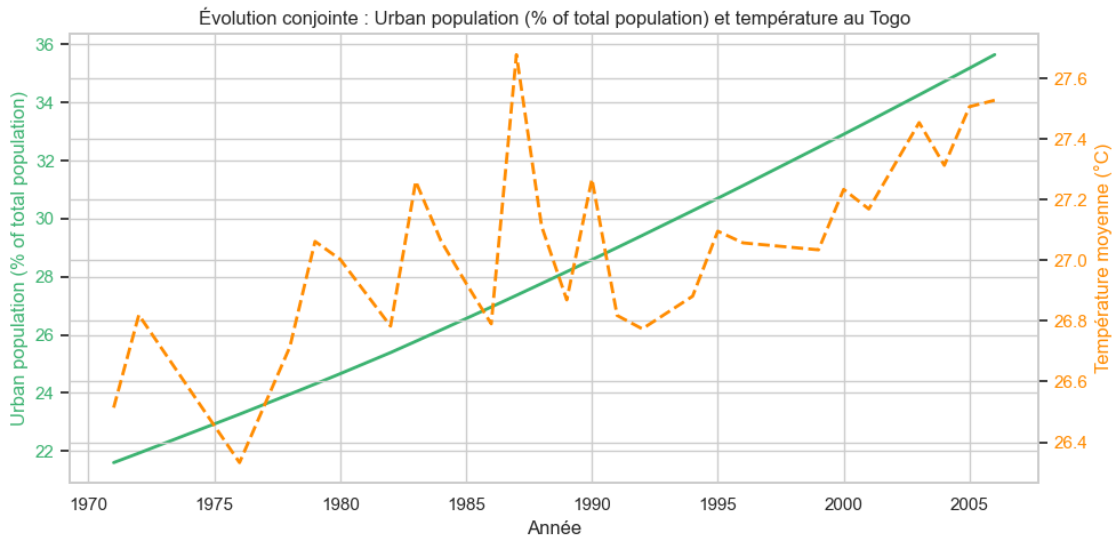



Évolution conjointe : Mortality rate, under-5 (per 1,000 live births) et température au Togo
Mortality rate, under-5 (per 1,000 live births)
Température moyenne (°C)
Année
220
200
180
160
140
120
100
27.6
27.4
27.2
27.0
26.8
26.6
26.4
1970
1975
1980
1985
1990
1995
2000
2005


Évolution conjointe : Population in urban agglomerations of more than 1 million (% of total population) et température au Togo
ation in urban agglomerations of more than 1 million (% of total po
Température moyenne (°C)
Année
22
20
18
16
14
12
10
27.6
27.4
27.2
27.0
26.8
26.6
26.4
1970
1975
1980
1985
1990
1995
2000
2005


Évolution conjointe : Primary completion rate, total (% of relevant age group) et température au Togo
Primary completion rate, total (% of relevant age group)
Température moyenne (°C)
Année
70
60
50
40
30
27.6
27.4
27.2
27.0
26.8
26.6
26.4
1970
1975
1980
1985
1990
1995
2000
2005


Évolution conjointe : School enrollment, primary and secondary (gross), gender parity index (GPI) et température au Togo
ool enrollment, primary and secondary (gross), gender parity inde
Température moyenne (°C)
Année
0.75
0.70
0.65
0.60
0.55
0.50
0.45
27.6
27.4
27.2
27.0
26.8
26.6
26.4
1970
1975
1980
1985
1990
1995
2000
2005



9.0.2 Hypothèse: La hausse des émissions de CO au Togo est liée à une amélioration des conditions de vie en milieu urbain, notamment en matière d'éducation, d'accès à l'électricité et de santé, mais elle s'accompagne d'une faible utilisation des énergies renouvelables.

[ ]:
```
# Filtrer uniquement le Togo
df_co2_togo = df_c02[df_c02["Country Name"] == "Togo"]

# Identifier les colonnes année (ex: "1960", "1961", ...)
colonnes_annees = [col for col in df_co2_togo.columns if col.isdigit()]

# Transformation en format long
df_co2_long = df_co2_togo.melt(
    id_vars=["Country Name", "Country Code", "Indicator Name", "Indicator␣
 ↪Code"],
    value_vars=colonnes_annees,
    var_name="Year",
    value_name="Value"
)

# Conversion de l'année en entier
df_co2_long["Year"] = pd.to_numeric(df_co2_long["Year"], errors="coerce")
df_co2_long = df_co2_long.dropna(subset=["Year"])
df_co2_long["Year"] = df_co2_long["Year"].astype(int)

# Filtrer l'indicateur CO par habitant
nom_indicateur_co2 = "Carbon dioxide (CO2) emissions excluding LULUCF per␣
 ↪capita (t CO2e/capita)"
```

```
df_co2_filtered = df_co2_long[df_co2_long["Indicator Name"] ==␣
↪nom_indicateur_co2]

# Renommer la colonne pour plus de clarté
df_co2_filtered = df_co2_filtered.rename(columns={"Value": "CO2_per_capita"})

# Garder uniquement les colonnes utiles
df_co2_filtered = df_co2_filtered[["Year", "CO2_per_capita"]].dropna()
```

[ ]:
```
# Liste des indicateurs de vie
indicateurs_vie = [
    'Access to electricity (% of population)',
    'Electric power consumption (kWh per capita)',
    'Renewable energy consumption (% of total final energy consumption)',
    'Mortality rate, under-5 (per 1,000 live births)',
    'Prevalence of underweight, weight for age (% of children under 5)',
    'Primary completion rate, total (% of relevant age group)',
    'School enrollment, primary and secondary (gross), gender parity index␣
↪(GPI)',
    'Poverty headcount ratio at $3.00 a day (2021 PPP) (% of population)',
    'Urban population (% of total population)',
    'Population in urban agglomerations of more than 1 million (% of total␣
↪population)',
    'Population living in areas where elevation is below 5 meters (% of total␣
↪population)',
    'Average precipitation in depth (mm per year)',
    'Droughts, floods, extreme temperatures (% of population, average␣
↪1990-2009)'
]

# Filtrer et pivoter
df_vie = df_togo[df_togo["Indicator Name"].isin(indicateurs_vie)]
df_vie_pivot = df_vie.pivot_table(index="Year", columns="Indicator Name",␣
↪values="Value")
df_vie_pivot.index = df_vie_pivot.index.astype(int)
```

[ ]:
```
# Restreindre aux années communes
annees_communes = df_vie_pivot.index.intersection(df_co2_filtered["Year"])
df_vie_pivot = df_vie_pivot.loc[annees_communes]
df_co2_filtered = df_co2_filtered[df_co2_filtered["Year"].isin(annees_communes)]

# Fusion
df_vie_co2 = df_vie_pivot.merge(df_co2_filtered, left_index=True,␣
↪right_on="Year")
df_vie_co2.set_index("Year", inplace=True)
```

[ ]:
```python
# Supprimer les colonnes trop vides (>80% de NaN)
df_vie_co2_clean = df_vie_co2.loc[:, df_vie_co2.isna().mean() < 0.8]

# Supprimer les lignes avec trop de NaN (max 5 valeurs manquantes)
df_vie_co2_clean = df_vie_co2_clean[df_vie_co2_clean.isna().sum(axis=1) <= 5]

# Vérification
print("Forme finale :", df_vie_co2_clean.shape)
print("Années disponibles :", df_vie_co2_clean.index.tolist())
df_vie_co2_clean.head()
```

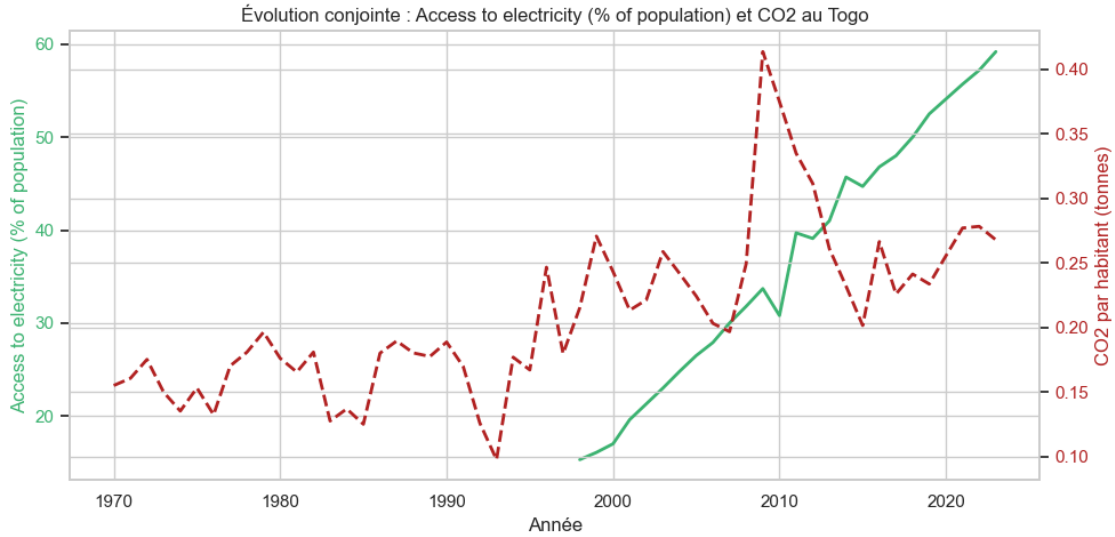
```
Forme finale : (54, 10)
Années disponibles : [1970, 1971, 1972, 1973, 1974, 1975, 1976, 1977, 1978,
1979, 1980, 1981, 1982, 1983, 1984, 1985, 1986, 1987, 1988, 1989, 1990, 1991,
1992, 1993, 1994, 1995, 1996, 1997, 1998, 1999, 2000, 2001, 2002, 2003, 2004,
2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013, 2014, 2015, 2016, 2017,
2018, 2019, 2020, 2021, 2022, 2023]
```

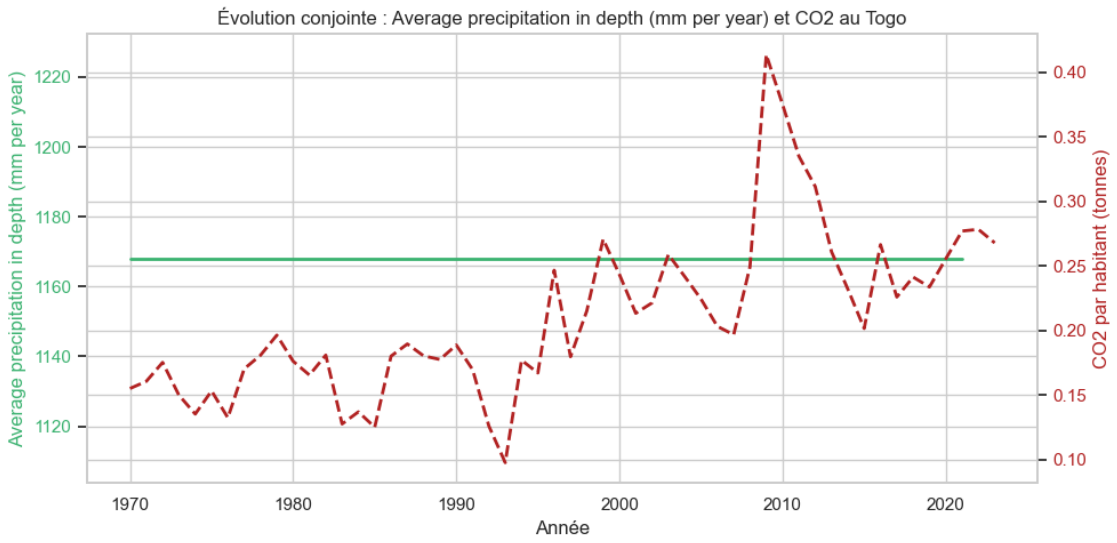
[ ]:
```
      Access to electricity (% of population)  \
Year
1970                                      NaN
1971                                      NaN
1972                                      NaN
1973                                      NaN
1974                                      NaN

      Average precipitation in depth (mm per year)  \
Year
1970                                         1168.0
1971                                         1168.0
1972                                         1168.0
1973                                         1168.0
1974                                         1168.0

      Electric power consumption (kWh per capita)  \
Year
1970                                          NaN
1971                                    60.397519
1972                                    57.861781
1973                                    52.271588
1974                                    67.283498

      Mortality rate, under-5 (per 1,000 live births)  \
Year
1970                                            225.2
1971                                            220.5
1972                                            215.9
1973                                            211.1
```

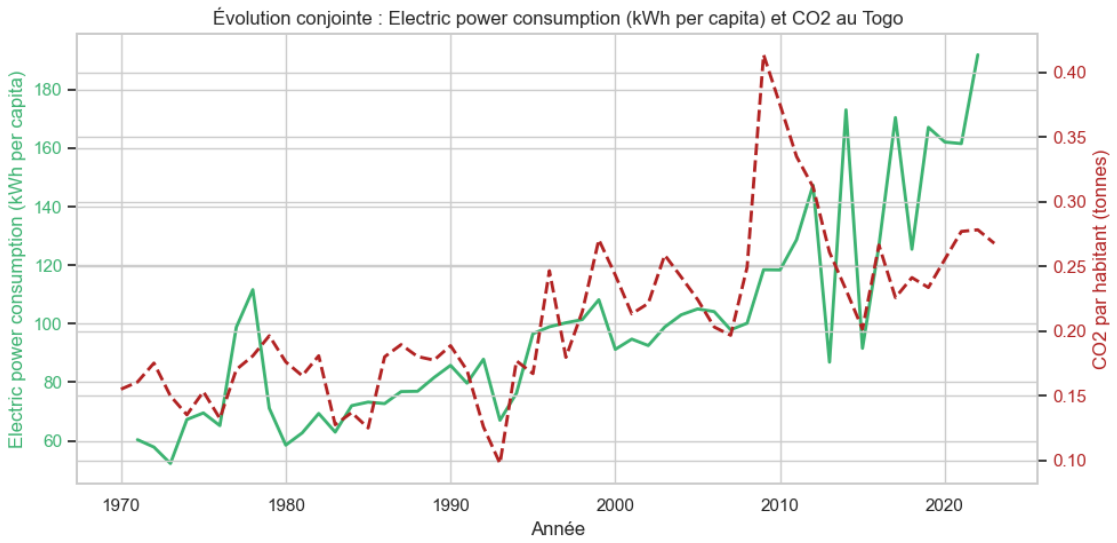
```
1974                                          206.5

      Population in urban agglomerations of more than 1 million (% of total
population)  \
Year
1970                                          8.537612
1971                                          8.765923
1972                                          9.033398
1973                                          9.327293
1974                                          9.640929

      Primary completion rate, total (% of relevant age group)  \
Year
1970                                                 NaN
1971                                           23.700991
1972                                           28.772610
1973                                                 NaN
1974                                                 NaN

      Renewable energy consumption (% of total final energy consumption)  \
Year
1970                                                 NaN
1971                                                 NaN
1972                                                 NaN
1973                                                 NaN
1974                                                 NaN

      School enrollment, primary and secondary (gross), gender parity index
(GPI)  \
Year
1970                                                 NaN
1971                                             0.43126
1972                                             0.44686
1973                                             0.45476
1974                                             0.46692

      Urban population (% of total population)  CO2_per_capita
Year
1970                                    21.280        0.155071
1971                                    21.602        0.160614
1972                                    21.929        0.175262
1973                                    22.258        0.149872
1974                                    22.591        0.135284
```

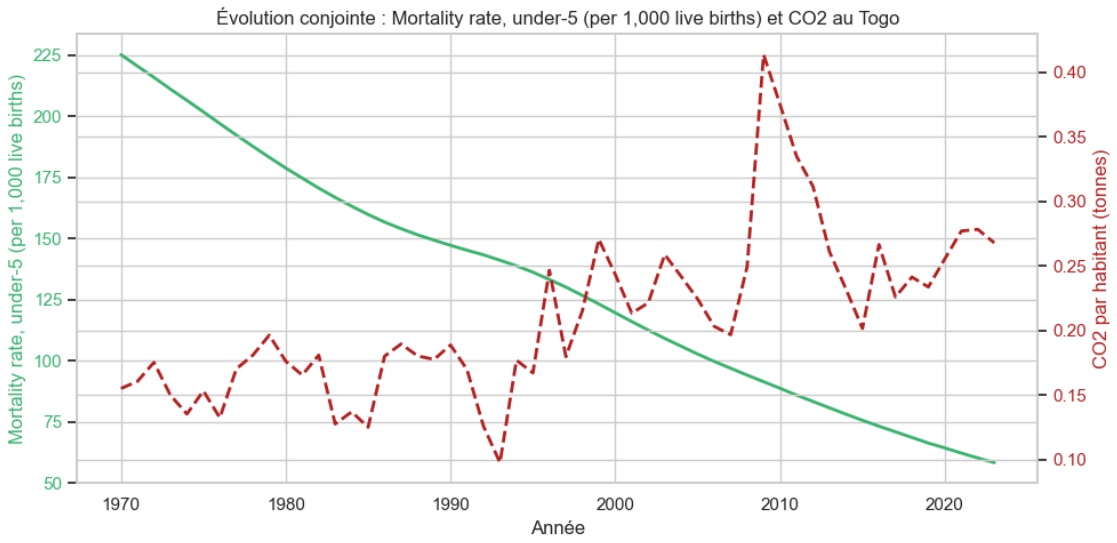
[ ]:
```
# Liste des indicateurs de vie (hors CO )
indicateurs_vie = [col for col in df_vie_co2_clean.columns if col !=␣
 ↪"CO2_per_capita"]
```

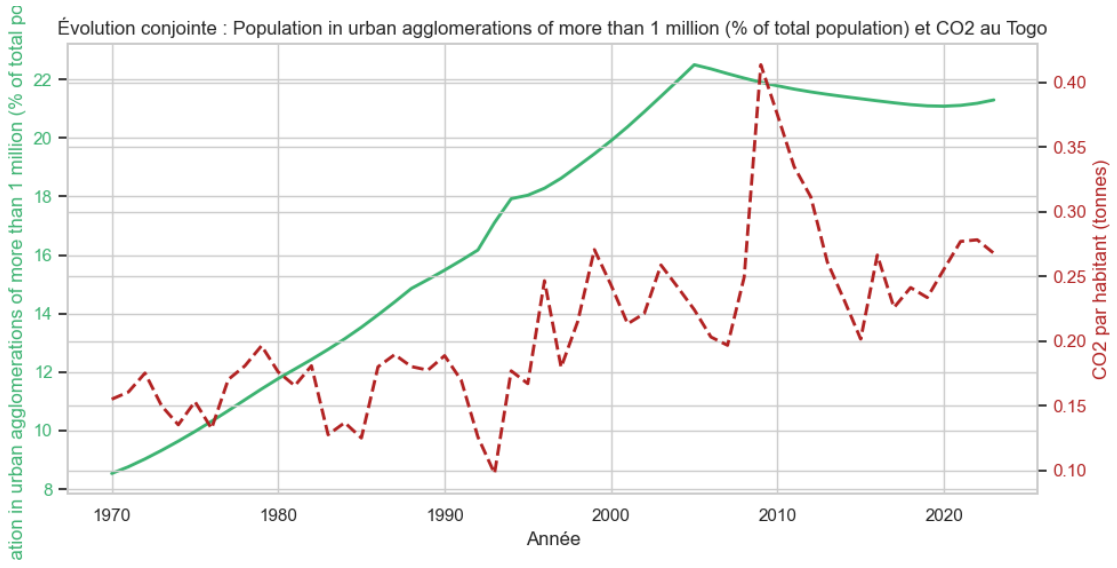
```
# Boucle de visualisation
for indicateur in indicateurs_vie:
    fig, ax1 = plt.subplots(figsize=(10, 5))

    # Axe Y gauche : indicateur de vie
    ax1.plot(df_vie_co2_clean.index, df_vie_co2_clean[indicateur],␣
↪color='mediumseagreen', linewidth=2)
    ax1.set_ylabel(indicateur, color='mediumseagreen')
    ax1.tick_params(axis='y', labelcolor='mediumseagreen')

    # Axe Y droit : CO  par habitant
    ax2 = ax1.twinx()
    ax2.plot(df_vie_co2_clean.index, df_vie_co2_clean["CO2_per_capita"],␣
↪color='firebrick', linestyle='--', linewidth=2)
    ax2.set_ylabel("CO2 par habitant (tonnes)", color='firebrick')
    ax2.tick_params(axis='y', labelcolor='firebrick')

    plt.title(f"Évolution conjointe : {indicateur} et CO2 au Togo")
    ax1.set_xlabel("Année")
    fig.tight_layout()
    plt.grid(True)
    plt.show()
```

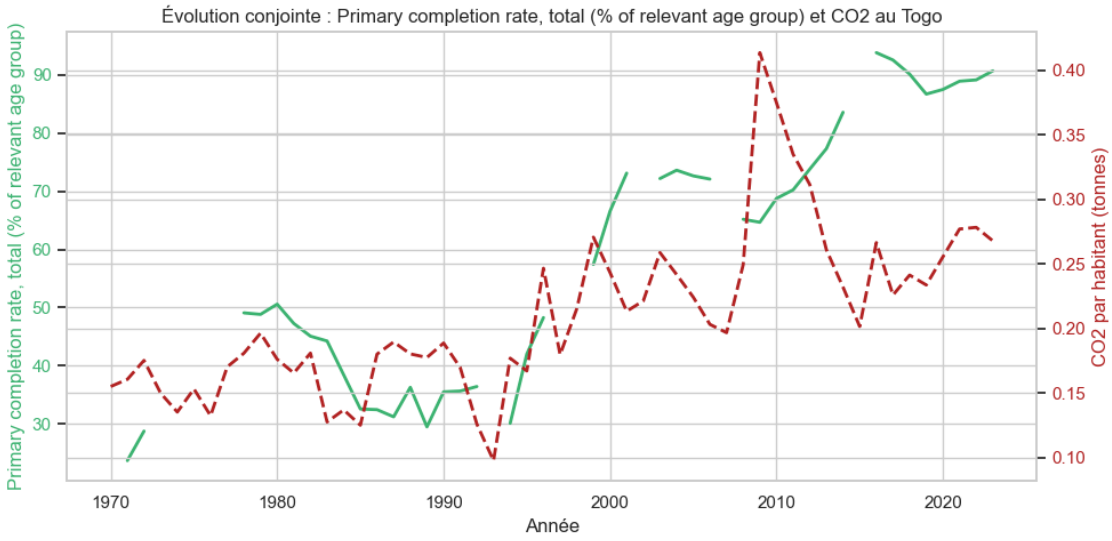


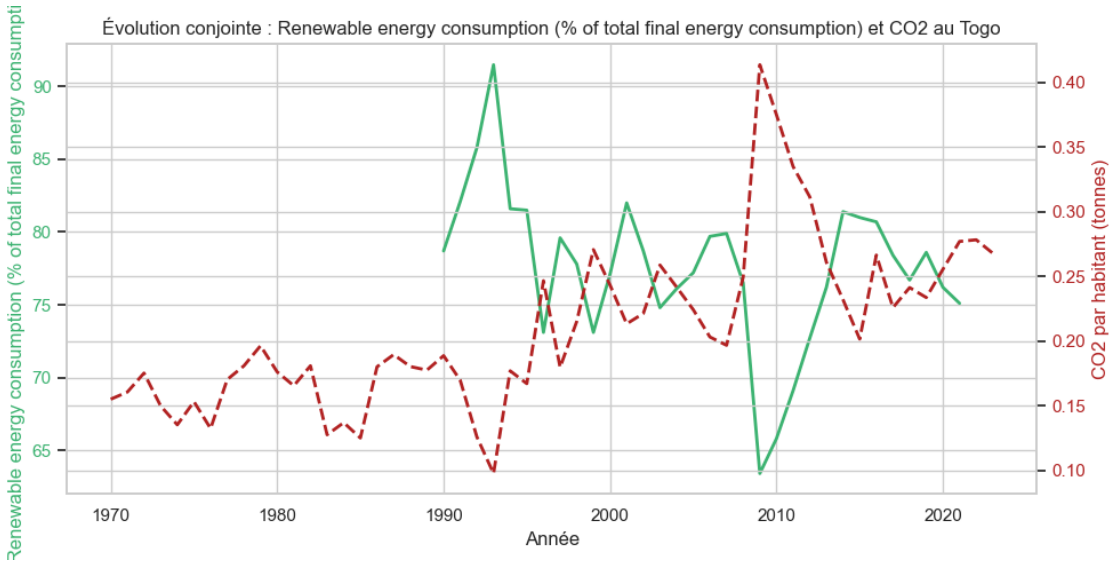

Évolution conjointe : Average precipitation in depth (mm per year) et CO2 au Togo
Average precipitation in depth (mm per year)
1220
1200
1180
1160
1140
1120
CO2 par habitant (tonnes)
0.40
0.35
0.30
0.25
0.20
0.15
0.10
1970
1980
1990
2000
2010
2020
Année

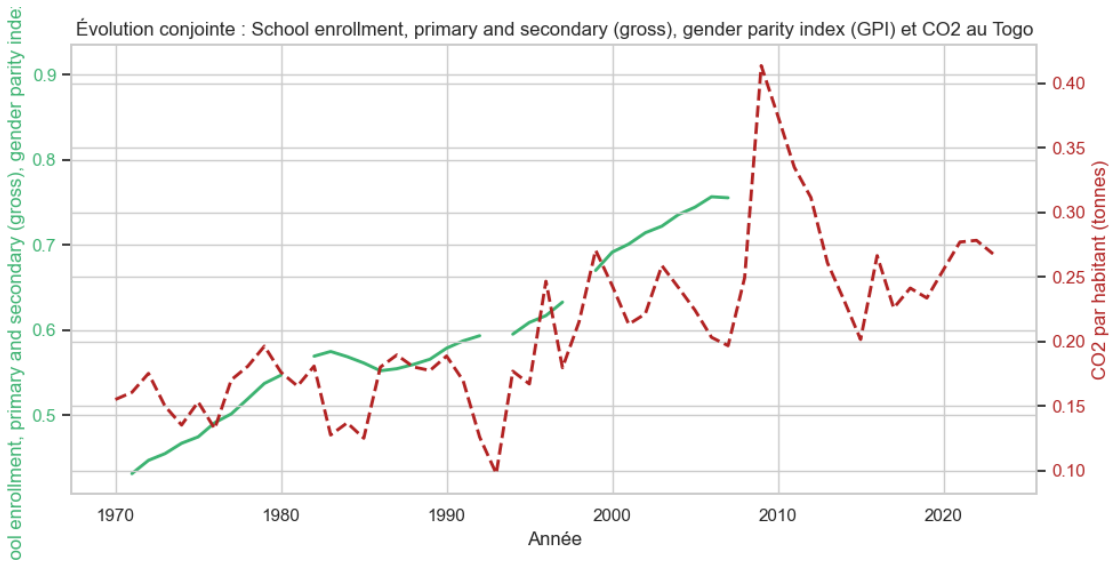

Évolution conjointe : Electric power consumption (kWh per capita) et CO2 au Togo
Electric power consumption (kWh per capita)
180
160
140
120
100
80
60
CO2 par habitant (tonnes)
0.40
0.35
0.30
0.25
0.20
0.15
0.10
1970
1980
1990
2000
2010
2020
Année

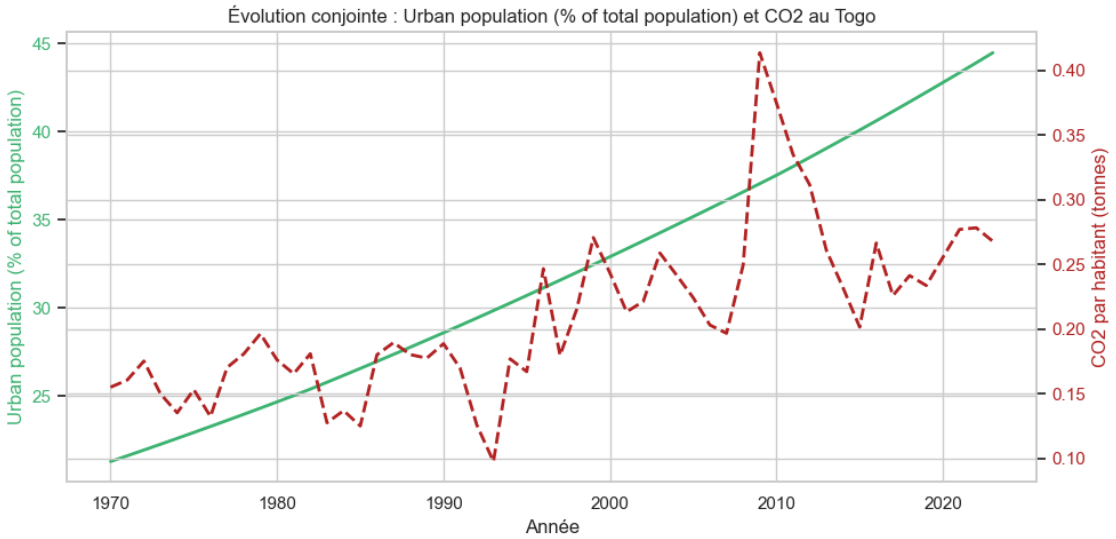

Évolution conjointe : Mortality rate, under-5 (per 1,000 live births) et CO2 au Togo
Mortality rate, under-5 (per 1,000 live births)
CO2 par habitant (tonnes)
Année
225
200
175
150
125
100
75
50
0.40
0.35
0.30
0.25
0.20
0.15
0.10
1970
1980
1990
2000
2010
2020


Évolution conjointe : Population in urban agglomerations of more than 1 million (% of total population) et CO2 au Togo
ation in urban agglomerations of more than 1 million (% of total po
CO2 par habitant (tonnes)
Année
22
20
18
16
14
12
10
8
0.40
0.35
0.30
0.25
0.20
0.15
0.10
1970
1980
1990
2000
2010
2020


Évolution conjointe : Primary completion rate, total (% of relevant age group) et CO2 au Togo
Primary completion rate, total (% of relevant age group)
CO2 par habitant (tonnes)
Année
90
80
70
60
50
40
30
0.40
0.35
0.30
0.25
0.20
0.15
0.10
1970
1980
1990
2000
2010
2020


Évolution conjointe : Renewable energy consumption (% of total final energy consumption) et CO2 au Togo
Renewable energy consumption (% of total final energy consumpti
CO2 par habitant (tonnes)
Année
90
85
80
75
70
65
0.40
0.35
0.30
0.25
0.20
0.15
0.10
1970
1980
1990
2000
2010
2020





9.0.3 Scolarisation (primaire + secondaire) vs CO

Observation

Scolarisation augmente fortement depuis 1970

CO reste faible, baisse légère dans certaines périodes

Interprétation

Togo investit dans l'éducation (campagnes alphabétisation 1975, UNESCO)

Aide internationale éducative forte

Croissance économique faible → industrie limitée → faible CO

Liens historiques

Période Événement Impact 1975–1985 Réforme scolaire + coopérations France/UNICEF hausse scolarisation 1990–1993 Crise politique/ grève générale stagnation scolarisation 2000–2010 Programmes Éducation Pour Tous forte augmentation → Togo se développe socialement sans industrialisation : éducation ↑ , CO ↓

9.0.4 Population urbaine (+1M hab) vs CO

Observation

Urbanisation augmente

CO monte mais très lentement

Explication

Croissance de Lomé, exode rural

Urbanisation = secteur tertiaire (commerce, port), pas industrie lourde

Histoire

Année Évènement 1968 Port de Lomé → urbanisation 1990 Crise → migration villes 2000s Infrastructures, routes, port modernisé Conclusion : urbanisation sans industrialisation → CO faible

9.0.5 Mortalité des enfants <5 ans vs CO

Observation

Mortalité enfant ↓↓↓ (200 80/1000)

CO fluctue légèrement

Explication

Progrès santé : vaccination, OMS, eau potable, campagnes nutrition

Recul des maladies infantiles

Événements

Période Mesures 1980–2000 Programmes OMS/UNICEF 2001 Initiative HIPC → fonds sociaux → Santé progresse même si économie faible

9.0.6 Population urbaine totale vs CO

Observation

% terres cultivées augmente

CO suit tendance mais reste bas

Analyse

Expansion agricoles (coton, maïs)

Défrichement rural

Histoire

Date Événement 1970–80 Politique agricole (autosuffisance) 1983 Ajustement FMI → agriculture priorité → agriculture ↑ industrie → CO pas explosif

[ ]:
```
correlations_co2 = {}

for indicateur in indicateurs_vie:
    corr = df_vie_co2_clean["CO2_per_capita"].corr(df_vie_co2_clean[indicateur])
    correlations_co2[indicateur] = round(corr, 3)

# Affichage
print(" Corrélations entre CO  par habitant et indicateurs de vie :")
for indicateur, corr in correlations_co2.items():
    print(f"- {indicateur} : {corr}")
```

```
 Corrélations entre CO  par habitant et indicateurs de vie :
- Access to electricity (% of population) : 0.129
- Average precipitation in depth (mm per year) : nan
- Electric power consumption (kWh per capita) : 0.634
- Mortality rate, under-5 (per 1,000 live births) : -0.701
- Population in urban agglomerations of more than 1 million (% of total
population) : 0.7
- Primary completion rate, total (% of relevant age group) : 0.653
- Renewable energy consumption (% of total final energy consumption) : -0.941
- School enrollment, primary and secondary (gross), gender parity index (GPI) :
0.725
- Urban population (% of total population) : 0.707
```

```
c:\Users\fangn\OneDrive\Bureau\BACHELOR 2 NEXA DIGITAL SCHOOL\SEMESTRE
1\EVOLUTION DU CLIMAT  CERVELLO\env\Lib\site-
packages\numpy\lib\_function_base_impl.py:3065: RuntimeWarning: invalid value
encountered in divide
  c /= stddev[:, None]
c:\Users\fangn\OneDrive\Bureau\BACHELOR 2 NEXA DIGITAL SCHOOL\SEMESTRE
1\EVOLUTION DU CLIMAT  CERVELLO\env\Lib\site-
packages\numpy\lib\_function_base_impl.py:3066: RuntimeWarning: invalid value
encountered in divide
  c /= stddev[None, :]
```

Electric power consumption (kWh per capita) +0.760 Plus les émissions augmentent, plus la consommation électrique progresse, signe d'un meilleur accès aux équipements et services.

Primary completion rate (% de l'âge concerné) +0.695 L'éducation s'améliore avec les émissions, ce qui reflète un développement social.

Urban population (% de la population totale) +0.741 L'urbanisation croît avec les émissions, ce qui montre une concentration du développement dans les villes.

Mortality rate, under-5 (taux de mortalité des enfants <5 ans) –0.699 La mortalité infantile diminue, ce qui indique une amélioration des conditions sanitaires.

Renewable energy consumption (% du total) –0.970 Corrélation très négative : les émissions augmentent quand la part des renouvelables diminue, ce qui montre une dépendance aux énergies polluantes.

Ces corrélations montrent que l'augmentation des émissions de CO au Togo est liée à une amélioration des conditions de vie, surtout en milieu urbain : plus d'électricité, meilleure éducation, baisse de la mortalité infantile. Mais en parallèle, la faible part des énergies renouvelables indique que ce progrès repose encore sur des sources polluantes, ce qui peut limiter la durabilité de ces avancées.